# 《云计算与微服务》实验指导书

## 实验名称、内容与学时分配表

| 序号 | 实验项目名称 | 实验简介 | 学时 | 实验类型 |
|---|---|---|---|---|
| 1 | 微服务开发环境搭建 | 内容：熟悉 IDEA、Maven、Git 的下载、安装及配合使用。<br>要求：学会 IDEA 的使用，掌握使用 IDEA 进行 Spring Boot 软件的开发，能够开发出简单的数据库基本操作 CURD 程序。 | 2 | 综合型 |
| 2 | 服务提供者 | 内容：创建父工程、提供者微服务和消费者微服务<br>要求：掌握开发提供者与消费者微服务模块的开发。 | 2 | 设计型 |
| 3 | 服务调用 | 内容：分别通过 restTemplate 和 OpenFeign 实现服务调用<br>要求：掌握服务调用的方法 | 2 | 设计型 |
| 4 | 负载均衡 | 内容：实现负载均衡<br>要求：掌握负载均衡的开发步骤 | 2 | 设计型 |
| 5 | 服务注册与发现 | 内容：服务实例在启动时注册到服务注册表，并在关闭时注销。服务和路由器的客户端查询服务注册表以查找服务的可用实例。服务注册中心可能会调用服务实例的健康检查 API 来验证它是否能够处理请求。<br>要求：掌握实现服务注册的方法，以及查看和管理微服务的基本操作。 | 2 | 设计型 |
| 6 | 分布式配置 | 内容：配置的编辑、存储、分发、变更管理、历史版本管理、变更审计的操作方法。<br>要求：掌握分布式配置管理的基本方法。 | 2 | 设计型 |
| 7 | 服务熔断 | 内容：通过 Sentinel 实现流量控制、熔断降级、系统负载保护等多个维度来保障服务之间的稳定性。<br>要求：重点掌握 Sentinel 的下载与运行、流利控制和熔断降级的实现方法。 | 2 | 设计型 |
| 8 | 服务网关 | 内容：通过 Gateway 实现服务网关功能。<br>要求：掌握服务网关的配置与开发。 | 2 | 设计型 |

| | | | | |
|---|---|---|---|---|
| 9 | Seata 事务管理 | 内容：了解什么是事务、事务的特征、事务的分类；单机模式下的事务和微服务模式下的事务的区别；学习 Seata 的使用<br>要求：对微服务下事务的概念有所了解，学会如何使用 Seata 实现分布式事务。 | 2 | 设计型 |
| 10 | 云计算与云部署 | 内容：将微服务应用打包，在 Docker 上部署<br>要求：掌握微服务应用打包、部署方法 | 2 | 综合型 |
| 11 附加 | 安全认证服务 | 内容：使用 Spring Security 完成项目的安全认证。<br>要求：掌握 Spring Security 的实现步骤。 | 2 | 设计型 |

【参考资料】

1. maven 仓库：https://mvnrepository.com/
2. 初始网站为：https://start.spring.io/ ，可以改为阿里的服务器：https://start.aliyun.com/
3. 注意：Spring Boot、Spring Cloud、Spring Cloud Alibaba 的版本可不是随便选择的，官网明确规定了各个版本的适配 https://github.com/alibaba/spring-cloud-alibaba/wiki/%E7%89%88%E6%9C%AC%E8%AF%B4%E6%98%8E

# 版本说明

Steve Rao edited this page on Jul 27, 2023 · 65 revisions

## 毕业版本依赖关系(推荐使用)

由于 Spring Boot 3.0，Spring Boot 2.7~2.4 和 2.4 以下版本之间变化较大，目前企业级客户老项目相关 Spring Boot 版本仍停留在 Spring Boot 2.4 以下，为了同时满足存量用户和新用户不同需求，社区以 Spring Boot 3.0 和 2.4 分别为分界线，同时维护 2022.x、2021.x、2.2.x 三个分支迭代。如果不想跨分支升级，如需使用新特性，请升级为对应分支的新版本。 为了规避相关构建过程中的依赖冲突问题，我们建议可以通过 云原生应用脚手架 进行项目创建。

### 2022.x 分支

适配 Spring Boot 3.0，Spring Cloud 2022.x 版本及以上的 Spring Cloud Alibaba 版本按从新到旧排列如下表（最新版本用*标记）：*(注意，该分支 Spring Cloud Alibaba 版本命名方式进行了调整，未来将对应 Spring Cloud 版本，前三位为 Spring Cloud 版本，最后一位为扩展版本，比如适配 Spring Cloud 2022.0.0 版本对应的 Spring Cloud Alibaba 第一个版本为：2022.0.0.0，第个二版本为：2022.0.0.1，依此类推)*

| Spring Cloud Alibaba Version | Spring Cloud Version | Spring Boot Version |
|---|---|---|
| 2022.0.0.0* | Spring Cloud 2022.0.0 | 3.0.2 |
| 2022.0.0.0-RC2 | Spring Cloud 2022.0.0 | 3.0.2 |
| 2022.0.0.0-RC1 | Spring Cloud 2022.0.0 | 3.0.0 |

## 2021.x 分支

适配 Spring Boot 2.4，Spring Cloud 2021.x 版本及以上的 Spring Cloud Alibaba 版本按从新到旧排列如下表（最新版本用*标记）：

| Spring Cloud Alibaba Version | Spring Cloud Version | Spring Boot Version |
| --- | --- | --- |
| 2021.0.5.0* | Spring Cloud 2021.0.5 | 2.6.13 |
| 2021.0.4.0 | Spring Cloud 2021.0.4 | 2.6.11 |
| 2021.0.1.0 | Spring Cloud 2021.0.1 | 2.6.3 |
| 2021.1 | Spring Cloud 2020.0.1 | 2.4.2 |

# 实验一 微服务开发环境搭建

【参考资料】

1、 https://www.bilibili.com/video/BV18a411q7iW/

2、 https://www.bilibili.com/video/BV1jJ411S7xr?p=4

3、https://www.bilibili.com/video/BV17o4y1o7VJ?p=6&spm_id_from=pageDriver

## 一、 目的与要求

掌握 IDEA、JDK、Maven、MySQL 的下载、安装、配置与使用。

## 二、 实验内容

2.1 安装配置 JDK。

2.2 安装配置 Maven。

2.3 熟悉 IDEA 安装、配置与开发项目的方法。

2.4 安装配置 MySQL 和 Navicat，创建练习用的数据库和表。

2.5 开发运行基于 Spring Boot 的“hello, world”。

## 三、 实验指导

【参考资料】软件开发环境安装教程

3.1 安装配置 JDK。

3.2 安装配置 Maven。

3.3 熟悉 IDEA 安装、配置与开发项目的方法。

IDEA 申请教育帐号的网址和步骤：

要申请 IDEA 的教育帐号，可以按照以下步骤进行：

- 前往 IDEA 官网的教育版申请页面，网址为：https://www.jetbrains.com/shop/eform/students
- 在申请页面，填写必要的信息，包括学校邮箱（必须是 xxx@edu.com 的形式）和其他相关信息。
- 提交申请后，会收到一封确认邮件。点击邮件中的链接，进入确认页面。
- 在确认页面，按照提示完成注册流程。
- 注册成功后，登录 IDEA IDE，选择 JBAccount，点击 Log in to JetBrains Account，填写账户名和密码，完成登录。

3.4 安装配置 MySQL 和 Navicat，创建练习用的数据库和表。

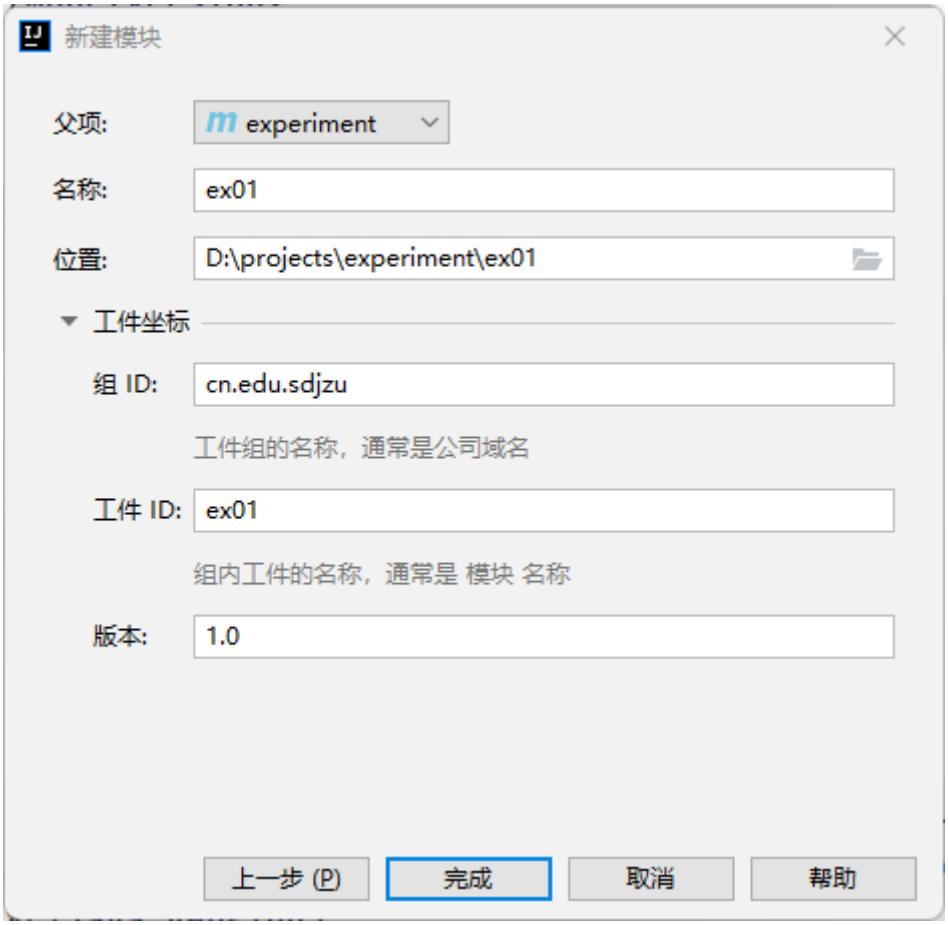
3.5 开发运行基于 Spring Boot 的“hello, world”。

1、新建 Maven 项目，项目名称为 experiment，该项目将作为父项目。删除 src 文件夹。

2、修改 pom.xml。建议直接从 Spring Cloud Alibaba Reference Documentation 复制并修改版本号。网址：https://spring-cloud-alibaba-group.github.io/github-pages/2021/en-us/index.html。或者直接复制下面代码。

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>cn.edu.sdjzu</groupId>
    <artifactId>experiment</artifactId>
    <version>1.0</version>

    <modules>
        <module>ex01</module>
    </modules>

    <!--打包方式-->
    <packaging>pom</packaging>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
        <java.version>1.8</java.version>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
        <spring.boot.version>2.6.4</spring.boot.version>
        <spring.cloud.version>2021.0.0</spring.cloud.version>
        <spring.cloud.alibaba.version>2021.0.1.0</spring.cloud.alibaba.version>
        <mysql.version>8.0.28</mysql.version>
        <mybatisplus.version>3.5.1</mybatisplus.version>
        <lombok.version>1.18.22</lombok.version>
        <spring.boot.devtools.version>2.6.6</spring.boot.devtools.version>
        <spring.boot.starter.web.version>2.6.4</spring.boot.starter.web.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <!--Spring Boot-->
            <dependency>
                <groupId>org.springframework.boot</groupId>
```

3、创建子模块 ex01。




【参考资料】

https://blog.csdn.net/m0_66782750/article/details/123980859

https://blog.csdn.net/m0_55734030/article/details/122440021

https://www.csdn.net/tags/NtjakgxsNDQ1ODItYmxvZwO0O0OO0O0O.html

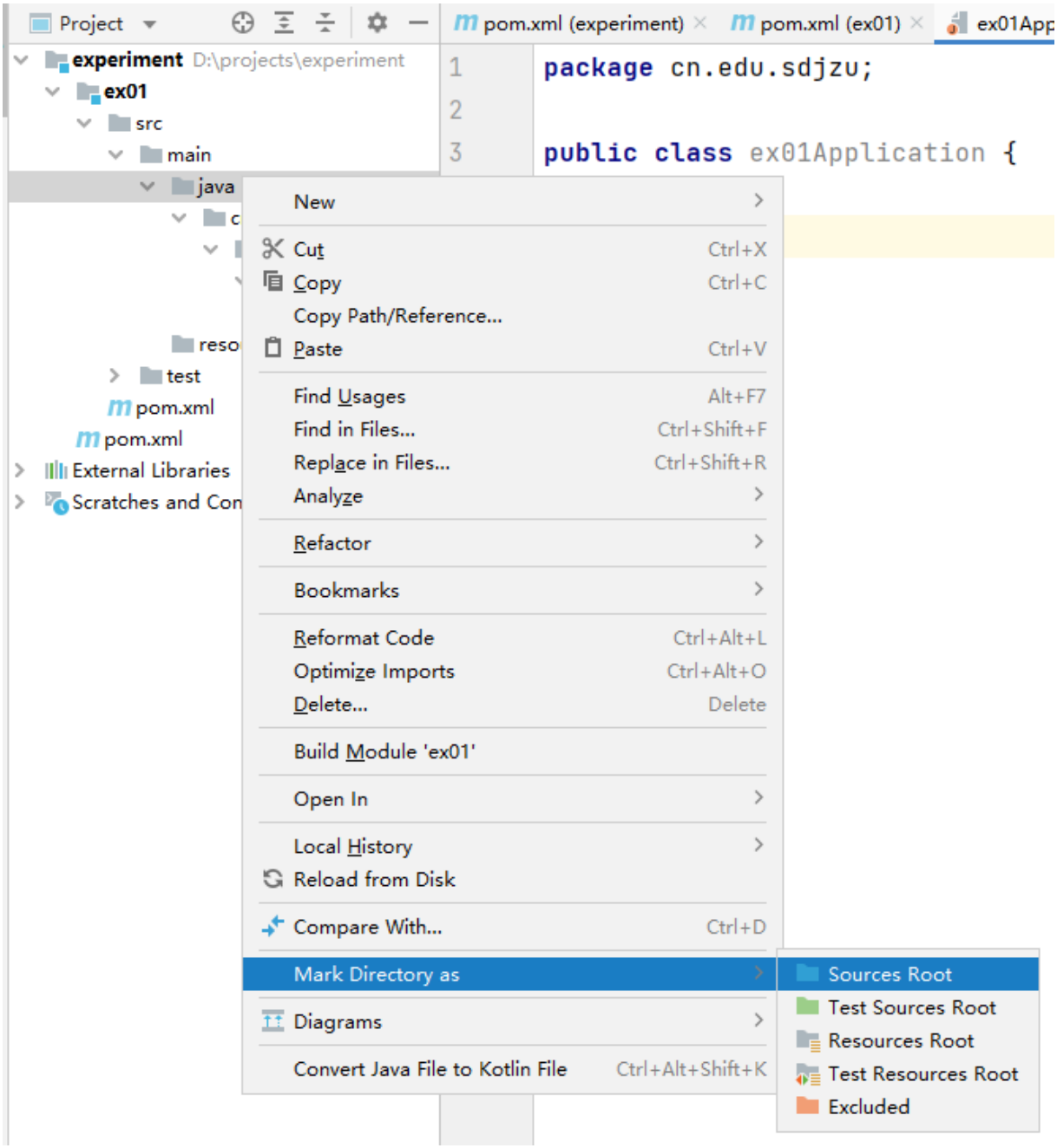
https://www.jb51.net/article/210005.htm

4、修改子模块的 pom.xml，建议直接复制修改以下代码。

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>experiment</artifactId>
        <groupId>cn.edu.sdjzu</groupId>
        <version>1.0</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>ex01</artifactId>

    <properties>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
    </properties>

    <dependencies>
        <!--web-->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <!--devtools-->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
        </dependency>
        <!--test-->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
        </dependency>
    </dependencies>

</project>
```

5、右击 java，创建包 Package，包名为 cn.edu.sdjzu。

如果这里出现问题，可以右击 java，选择 Mark Directory As→Sources Root




然后这里 java 由 java 变成 java，现在就可以创建包 Package 了。

6、在包 cn.edu.sdjzu 下创建 ex01Application.java

```
package cn.edu.sdjzu;

public class ex01Application {
    public static void main(String[] args) {
        System.out.println("hello，  world");
    }
}
```

运行该代码，则会在控制台输出 hello, world。

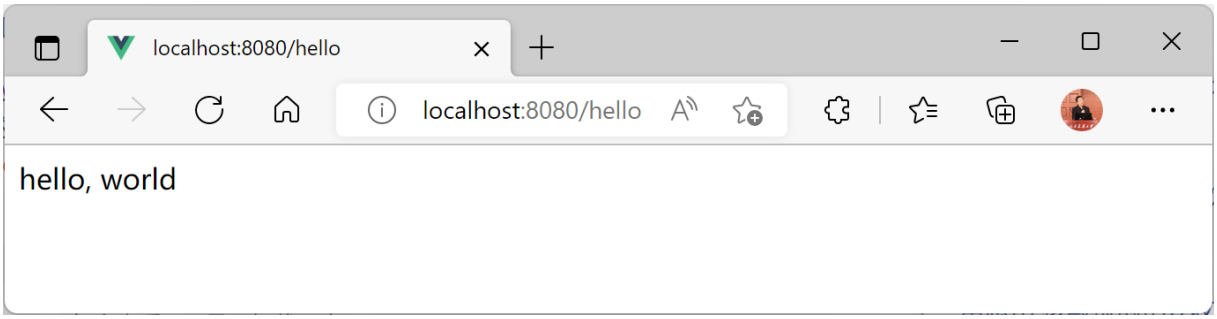
7、接下来我们创建 web 版的 hello, world。修改 pom.xml，添加 web 依赖。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

8、创建包 controller，创建 HelloController.java。

```
package cn.edu.sdjzu.experiment.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    @GetMapping("/hello")
    public String Hello() {
        return "hello, world";
    }
}
```

9、在浏览器输入：http://localhost:8080/hello，显示输出结果如下


localhost:8080/hello
localhost:8080/hello
hello, world

# 实验二 服务提供者与消费者

【参考资料】

1、 https://www.bilibili.com/video/BV17o4y1o7VJ?p=8&spm_id_from=pageDriver

## 一、 目的与要求

掌握创建父工程和子模块的方法。

## 二、 实验内容

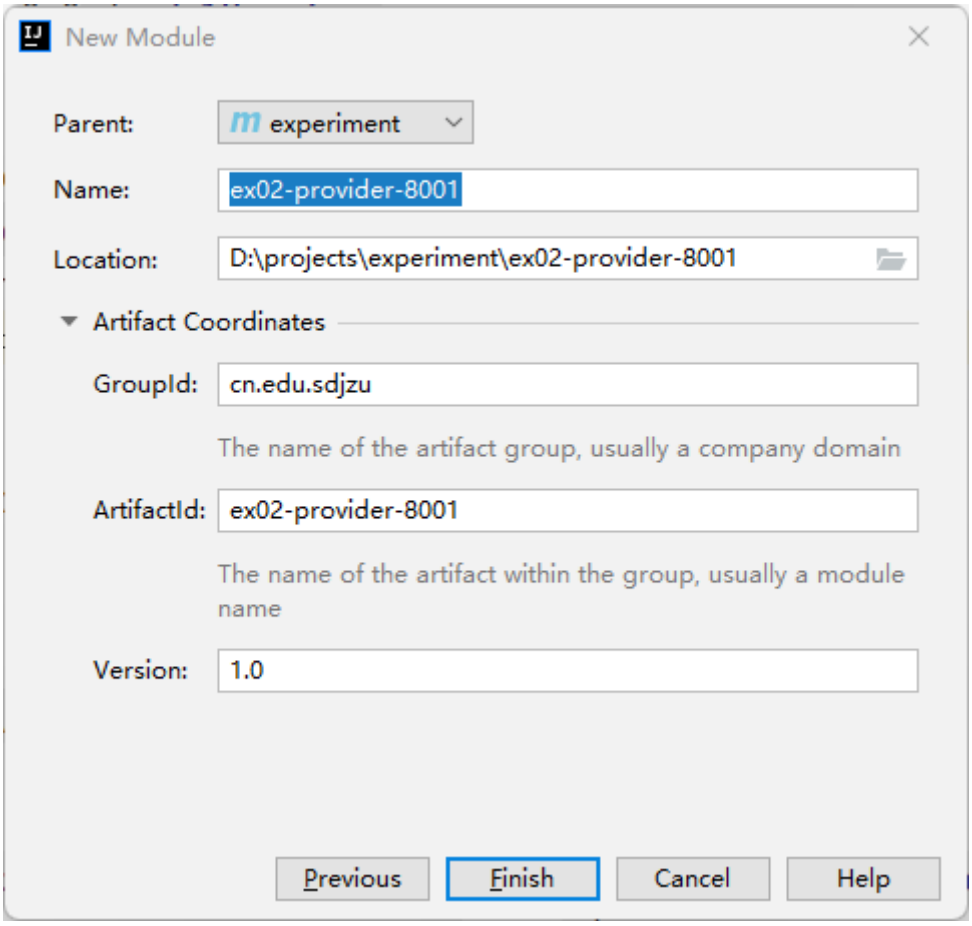
2.1 创建父工程

2.2 创建服务提供者子模块

2.3 创建服务消费者子模块

## 三、 实验指导

1、 新建 Maven 项目 ex02-provider-8001



2、修改 pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <parent>
        <artifactId>experiment</artifactId>
        <groupId>cn.edu.sdjzu</groupId>
        <version>1.0</version>
    </parent>
    <modelVersion>4.0.0</modelVersion>

    <artifactId>ex02-provider-8001</artifactId>

    <properties>
        <maven.compiler.source>8</maven.compiler.source>
        <maven.compiler.target>8</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
        </dependency>
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
        </dependency>
        <dependency>
            <groupId>com.baomidou</groupId>
            <artifactId>mybatis-plus-boot-starter</artifactId>
            <version>3.5.1</version>
        </dependency>
    </dependencies>

</project>
```

3、新建 application.yml

注意：url:

jdbc:mysql://localhost:3306/xgk2022?characterEncoding=utf8&useSSL=false&serverTimezone=Asia/Shanghai

xgk2022 是数据库名，大家在开发时，换成你自己的数据库名称。

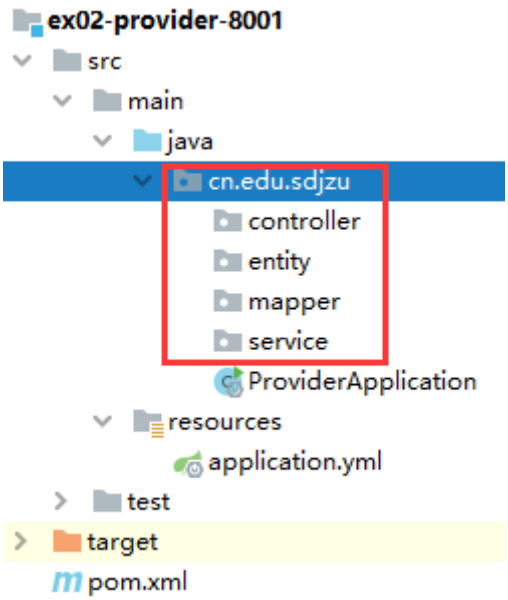
serverTimezone=Asia/Shanghai：这个是时区设置

```
server:
  port: 8001
spring:
  application:
    name: provider
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url:
jdbc:mysql://localhost:3306/xgk2022?characterEncoding=utf8&useSSL=false&serverTimezone=Asia/Shanghai
    username: root
    password: root
```

4、新建 ProviderApplication.java

```
package cn.edu.sdjzu;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ProviderApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProviderApplication.class, args);
    }
}
```

5、分别创建以下包




6、entity

University.java（文件内容见下一实验）

7、mapper

```
package cn.edu.sdjzu.mapper;

import cn.edu.sdjzu.entity.University;
import com.baomidou.mybatisplus.core.mapper.BaseMapper;

public interface UniversityMapper extends BaseMapper<University> {

}
```

UniversityMapper.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
"http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="cn.edu.sdjzu.expert.mapper.UniversityMapper">

</mapper>
```

8、service

IUniversityService.java

```
package cn.edu.sdjzu.service;

import cn.edu.sdjzu.entity.University;
import com.baomidou.mybatisplus.extension.service.IService;

public interface IUniversityService extends IService<University> {

}
```

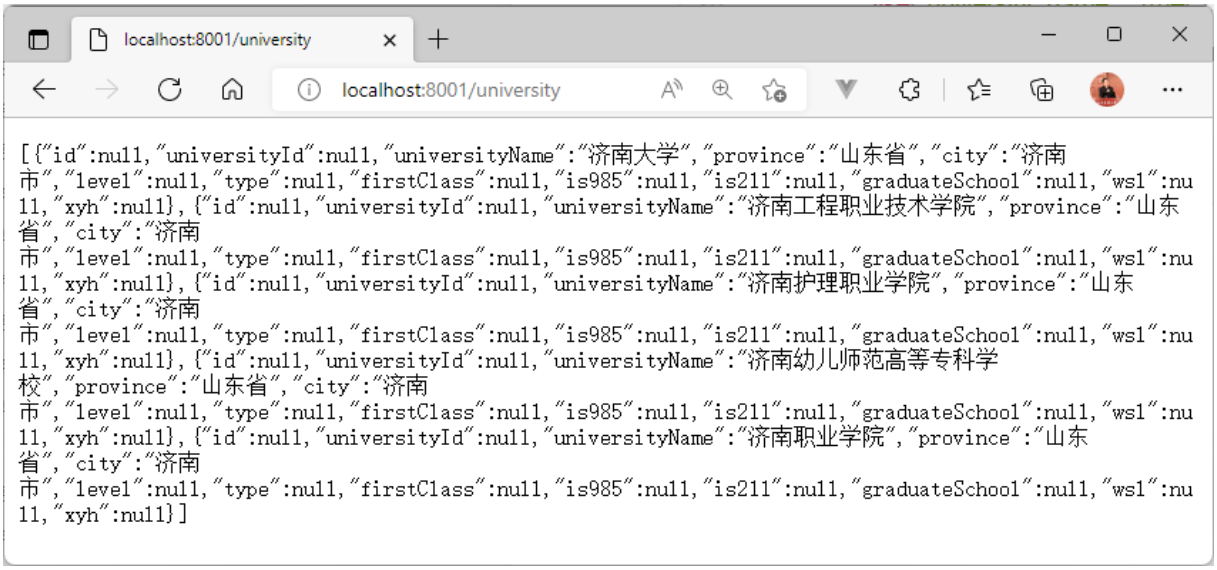
UniversityServiceImpl.java

```
package cn.edu.sdjzu.service.impl;

import cn.edu.sdjzu.entity.University;
import cn.edu.sdjzu.mapper.UniversityMapper;
import cn.edu.sdjzu.service.IUniversityService;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import org.springframework.stereotype.Service;

@Service
public class UniversityServiceImpl extends ServiceImpl<UniversityMapper, University>
implements IUniversityService {

}
```

9、controller

```
package cn.edu.sdjzu.controller;

import cn.edu.sdjzu.entity.University;
import cn.edu.sdjzu.service.IUniversityService;
import
com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class UniversityController {
    private final IUniversityService iUniversityService;

    public UniversityController(IUniversityService
iUniversityService) {
        this.iUniversityService = iUniversityService;
    }

    @GetMapping("/university")
    public List<University> selectList() {
        QueryWrapper<University> queryWrapper = new
QueryWrapper<>();
        queryWrapper
                .select("university_name", "province", "city")
                .like("university_name", "济南");
        return iUniversityService.list(queryWrapper);
    }
}
```

10、 运行




[{"id":null,"universityId":null,"universityName":"济南大学","province":"山东省","city":"济南
市","level":null,"type":null,"firstClass":null,"is985":null,"is211":null,"graduateSchool":null,"wsl":nu
ll,"xyh":null},{"id":null,"universityId":null,"universityName":"济南工程职业技术学院","province":"山东
省","city":"济南
市","level":null,"type":null,"firstClass":null,"is985":null,"is211":null,"graduateSchool":null,"wsl":nu
ll,"xyh":null},{"id":null,"universityId":null,"universityName":"济南护理职业学院","province":"山东
省","city":"济南
市","level":null,"type":null,"firstClass":null,"is985":null,"is211":null,"graduateSchool":null,"wsl":nu
ll,"xyh":null},{"id":null,"universityId":null,"universityName":"济南幼儿师范高等专科学
校","province":"山东省","city":"济南
市","level":null,"type":null,"firstClass":null,"is985":null,"is211":null,"graduateSchool":null,"wsl":nu
ll,"xyh":null},{"id":null,"universityId":null,"universityName":"济南职业学院","province":"山东
省","city":"济南
市","level":null,"type":null,"firstClass":null,"is985":null,"is211":null,"graduateSchool":null,"wsl":nu
ll,"xyh":null}]

# 实验三 服务调用

## 一、 目的与要求

掌握远程服务调用的方法。

## 二、 实验内容

通过 restTemplate 实现远程调用。

## 三、 实验指导

1、因为服务提供者和服务消费者都需要用到实体类 University.java，所以需要创建一个公共模块 common。

在 pom.xml 中添加依赖：

```
<dependencies>
    <dependency>
        <groupId>com.baomidou</groupId>
        <artifactId>mybatis-plus-boot-starter</artifactId>
        <version>3.5.1</version>
    </dependency>
</dependencies>
```

新建包 cn.edu.sdjzu.entity，创建实体类 University.java，其中第 17-19 行，要重点留意：

```
17  public class University implements Serializable {
18
19      private static final long serialVersionUID = 1L;
20
```

【参考资料】Java 序列化和反序列化，为什么要实现 Serializable 接口？

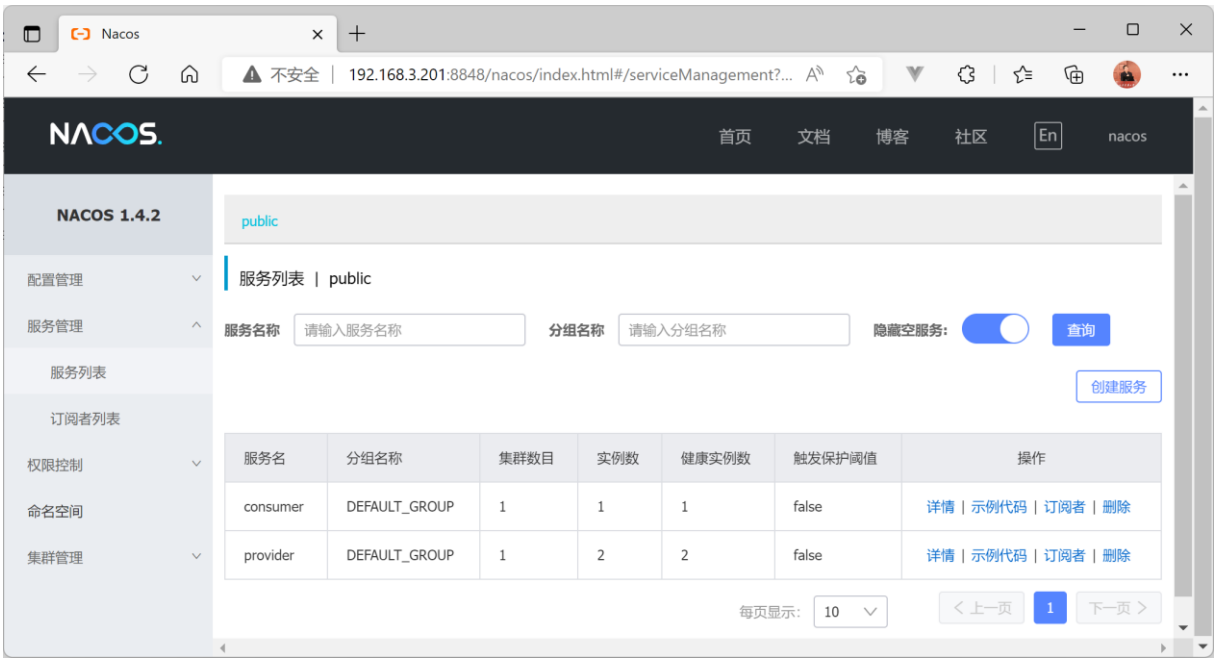
https://mp.weixin.qq.com/s?__biz=MzU5MDgyMzc2OA==&mid=2247515720&idx=2&sn=f94e8c35805871db45333f5cb01f775d&chksm=fe3a967fc94d1f691a93c4cdc202894fe191774baf58495d218efc6a3ec742f189ea333afdf1&mpshare=1&scene=23&srcid=0607XSxbE2MSvW11jAgakuM6&sharer_sharetime=1655423947363&sharer_shareid=a9e65e915ecf3a178d99b6dd0b5e1cfc#rd

```java
package cn.edu.sdjzu.entity;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;

import java.io.Serializable;

/**
 * <p>
 *
 * </p>
 *
 * @author 杨磊
 * @since 2022年06月02日
 */
public class University implements Serializable {

    private static final long serialVersionUID = 1L;

    /**
     * 编号
     */
    @TableId(value = "id", type = IdType.AUTO)
    private Long id;

    /**
     * 院校代码
     */
    private String universityId;

    /**
     * 院校名称
     */
    private String universityName;

    /**
     * 省份
     */
    private String province;

    /**
```

2、在服务提供者和服务消费者模块都需要修改 pom.xml，以调用公共模块 common 中的实体类 University。注意：该例中，公共模块命名为 ex02-common。

```
<dependency>
    <groupId>cn.edu.sdjzu</groupId>
    <artifactId>ex02-common</artifactId>
    <version>1.0</version>
</dependency>
```

3、新建服务消费者模块 ex03-consumer-8002，文件清单如下。为了实现远程通过服务名进行调用，需要通过 Nacos 进等服务注册。另外，由于新版的 Nacos 不再内置 rabbin，所以采用替代方案，添加 loadbalancer 依赖，实现远程调用和负载均衡。【关于 Nacos 服务注册，详细步骤见实验五 服务注册与发现】




pom.xml

```
    <dependencies>
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
        </dependency>
        <dependency>
            <groupId>com.baomidou</groupId>
            <artifactId>mybatis-plus-boot-starter</artifactId>
            <version>3.5.1</version>
        </dependency>
        <dependency>
            <groupId>cn.edu.sdjzu</groupId>
            <artifactId>ex02-common</artifactId>
            <version>1.0</version>
        </dependency>
        <!--
    https://mvnrepository.com/artifact/com.alibaba.cloud/spring-
    cloud-starter-alibaba-nacos-discovery -->
        <dependency>
            <groupId>com.alibaba.cloud</groupId>
            <artifactId>spring-cloud-starter-alibaba-nacos-
    discovery</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-
    loadbalancer</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        </dependencies>
```

application.yml：这里要留意多个模块同时运行，造成 devtools 端口冲突的问题

```
server:
  port: 8002
spring:
  application:
    name: consumer
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url:
jdbc:mysql://localhost:3306/xgk2022v3?characterEncoding=utf8&useS
SL=false&serverTimezone=Asia/Shanghai
    username: root
    password: root
  cloud:
    nacos:
      discovery:
        server-addr: 127.0.0.1:8848
  devtools:
    livereload:
      port: 35731
```

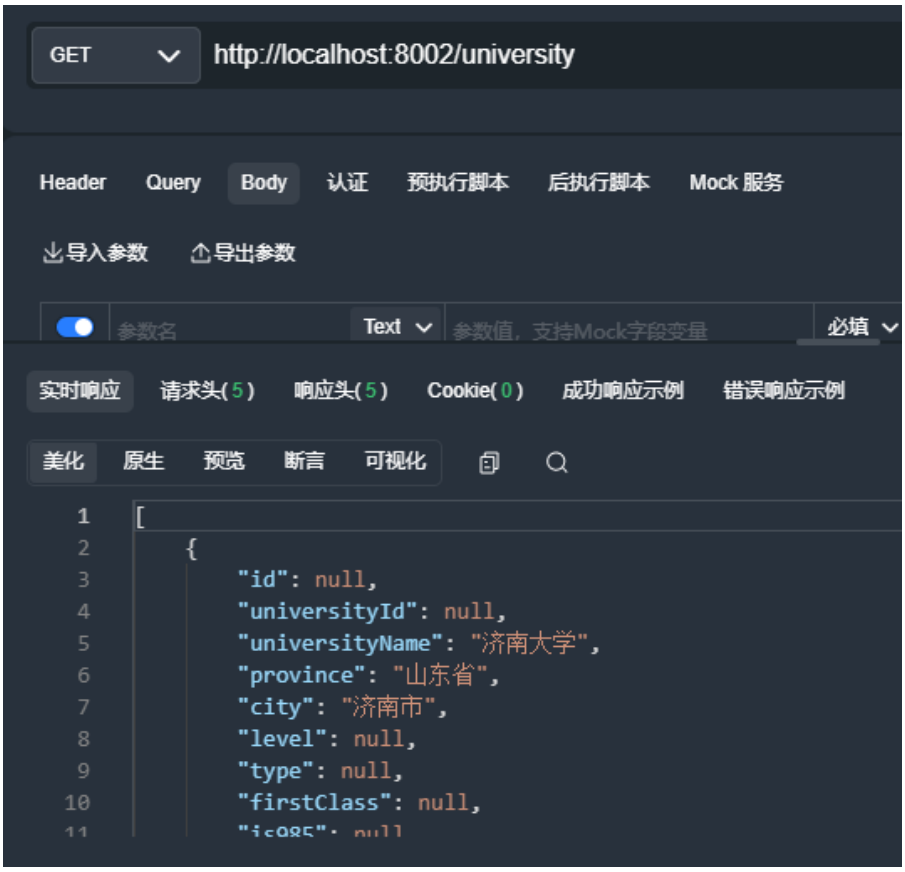
ConsumerApplication.java

```
@SpringBootApplication
public class ConsumerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConsumerApplication.class, args);
    }

    @Bean
    @LoadBalanced
    public RestTemplate getRestTemplate() {
        return new RestTemplate();
    }
}
```

UniversityController.java

```
@RestController
public class UniversityController {
    @Autowired
    private RestTemplate restTemplate;

    public static String url = "http://provider/";

    @GetMapping("/university")
    public List<University> selectList() {
        return restTemplate.getForObject(url + "university",
List.class);
    }
}
```

运行结果：



# 实验四 负载均衡

## 一、 目的与要求

掌握负载均衡的配置实现方法。再加上端口输出部分内容：

@Value("${server.port}")

String port

https://mp.weixin.qq.com/s?__biz=MzU3MDAzNDg1MA==&mid=2247493854&idx=1&sn=4b3fb7f7e17a76000733899f511ef915&scene=21#wechat_redirect

## 二、 实验内容

【视频教程】https://hikeweb.zhihuishu.com/studyResource/index?courseId=10400917 智慧树：杨老师-云计算与微服务，课程号：k745155

【视频教程】OpenFeign 快速使用

11:33https://www.bilibili.com/video/BV1KY4y1p7zZ?p=27&vd_source=c5e65f020919c07bb4d1cc902ee3c1d7

2.1 通过 OpenFeign 实现远程调用

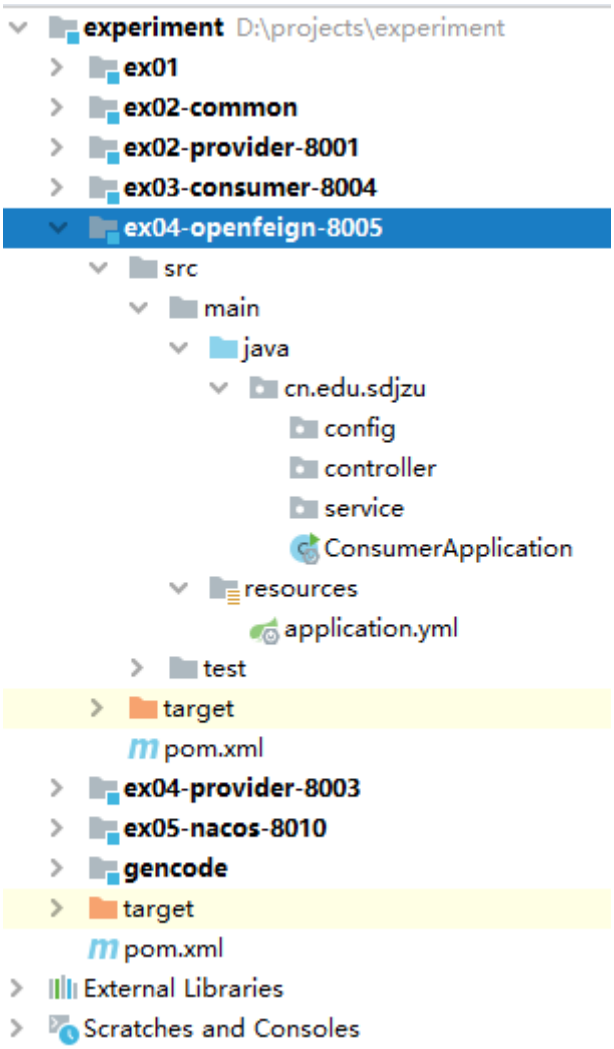
【参考资料】https://spring.io/projects/spring-cloud-openfeign

2.2 通过 rabbin 实现基本的轮询负载均衡算法

【参考资料】微服务简明教程 20220204V1

## 三、 实验指导

1、新建子模块




2、修改 pom.xml，添加依赖

```
<dependencies>
    <dependency>
        <groupId>cn.edu.sdjzu</groupId>
        <artifactId>ex02-common</artifactId>
        <version>1.0</version>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-openfeign</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-
loadbalancer</artifactId>
    </dependency>

    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
    </dependency>

    <dependency>
        <groupId>com.alibaba.cloud</groupId>
        <artifactId>spring-cloud-starter-alibaba-nacos-
discovery</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
    </dependency>
</dependencies>
```

3、新建 application.yml

```
server:
  port: 8005
spring:
  application:
    name: consumer-openfeign-8005
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url:
jdbc:mysql://localhost:3306/xgk2022v3?characterEncoding=utf8&useS
SL=false&serverTimezone=Asia/Shanghai
    username: root
    password: root
  devtools:
    livereload:
      port: 35735
```

4、新建主程序

这里要加上两个关键注解：

`@ComponentScan("cn.edu.sdjzu.*")`

`@EnableFeignClients`，用来开启 Feign

```
package cn.edu.sdjzu;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;
import org.springframework.context.annotation.ComponentScan;

@SpringBootApplication
@ComponentScan("cn.edu.sdjzu.*")
@EnableFeignClients
public class ConsumerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConsumerApplication.class, args);
    }
}
```

5、关键代码：新建 service 包，创接接口 UniversityService

这里要加上的两个关键注解：

@Component

@FeignClient(name = "provider", path = ""），标记要用 Feign 来拦截的请求接口。

这里 path=""，是对应下图中的第 13 行中的/provider。如果没有，保持为空即可。

```
10  import java.util.List;
11
12  @RestController
13  @RequestMapping(value="/provider")
14  public class ApplicationController {
```

```
package cn.edu.sdjzu.service;

import cn.edu.sdjzu.entity.University;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.stereotype.Component;
import org.springframework.web.bind.annotation.GetMapping;

import java.util.List;

@Component
@FeignClient(name = "provider", path = "")
public interface UniversityService {
    @GetMapping("/university")
    List<University> selectList();
}
```

6、控制器 UniversityController.java

```
package cn.edu.sdjzu.controller;

import cn.edu.sdjzu.entity.University;
import cn.edu.sdjzu.service.UniversityService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;
@RestController
public class UniversityController {
    @Autowired
    private UniversityService universityService;
    @GetMapping("/university")
    public List<University> selectList() {
        return universityService.selectList();
    };
}
```

7、运行情况如下：

8、配置文件 FeignConfig.java

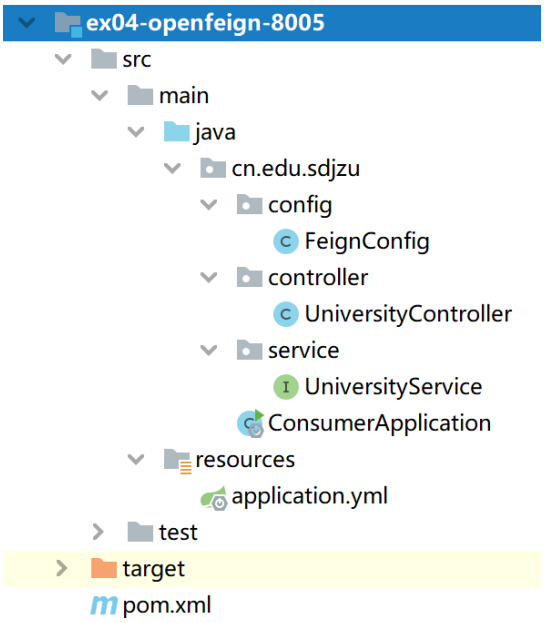
对于响应超时问题，可以进行配置处理，代码如下：

```
package cn.edu.sdjzu.config;

import feign.Request;
import org.springframework.context.annotation.Configuration;

@Configuration
public class FeignConfig {
    public Request.Options setTime() {
        return new Request.Options(5000, 10000);
    }
}
```

这个也可以不配置，直接使用默念超时配置。

```
@Deprecated
public Options(int connectTimeoutMillis, int readTimeoutMillis) {
    this(connectTimeoutMillis, readTimeoutMillis, true);
}
```

这里两个参考 的含义分别是连接超时、读超时。最终的项目结构如下：


ex04-openfeign-8005
src
main
java
cn.edu.sdjzu
config
FeignConfig
controller
UniversityController
service
UniversityService
ConsumerApplication
resources
application.yml
test
target
pom.xml

# 实验五 服务注册与发现

【参考资料】

1、 https://www.jianshu.com/p/c3f810cbe061

2、 https://blog.csdn.net/qq_38050728/article/details/121924621

3、 https://nacos.io/zh-cn/docs/quick-start.html

## 一、 目的与要求

掌握 Nacos 实现服务注册的方法。

## 二、 实验内容

2.1 下载并运行 Nacos

2.2 编写一微服务用例实现服务注册

## 三、 实验指导

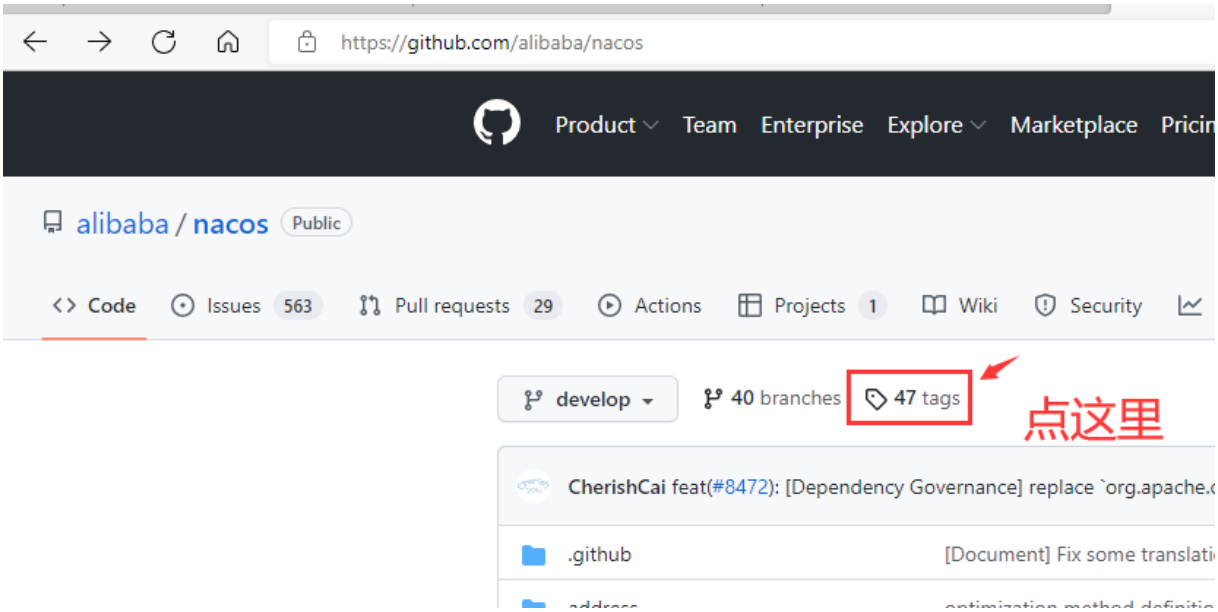
1、下载 Nacos

1.1 Nacos 官网：https://nacos.io/

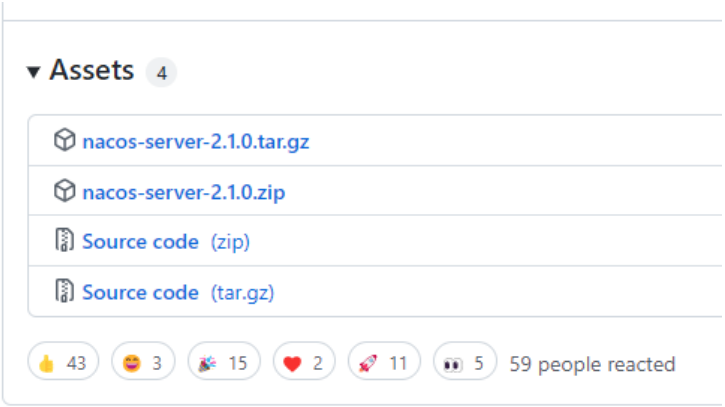
【参考资料】Nacos 2.1.0 正式发布！堪称最强！

https://mp.weixin.qq.com/s?__biz=MzAxNDMwMTMwMw==&mid=2247538010&idx=2&sn=cd1ddeff8aa547fa5395705179fee8e0&chksm=9b977442ace0fd5494993141005748fe616e38ed7bde148a1c4274550a791fbb6cfa7c4a86d8&mpshare=1&scene=23&srcid=05169qRZgDne5NxZtBPBnxct&sharer_sharetime=1655423973335&sharer_shareid=a9e65e915ecf3a178d99b6dd0b5e1cfc#rd

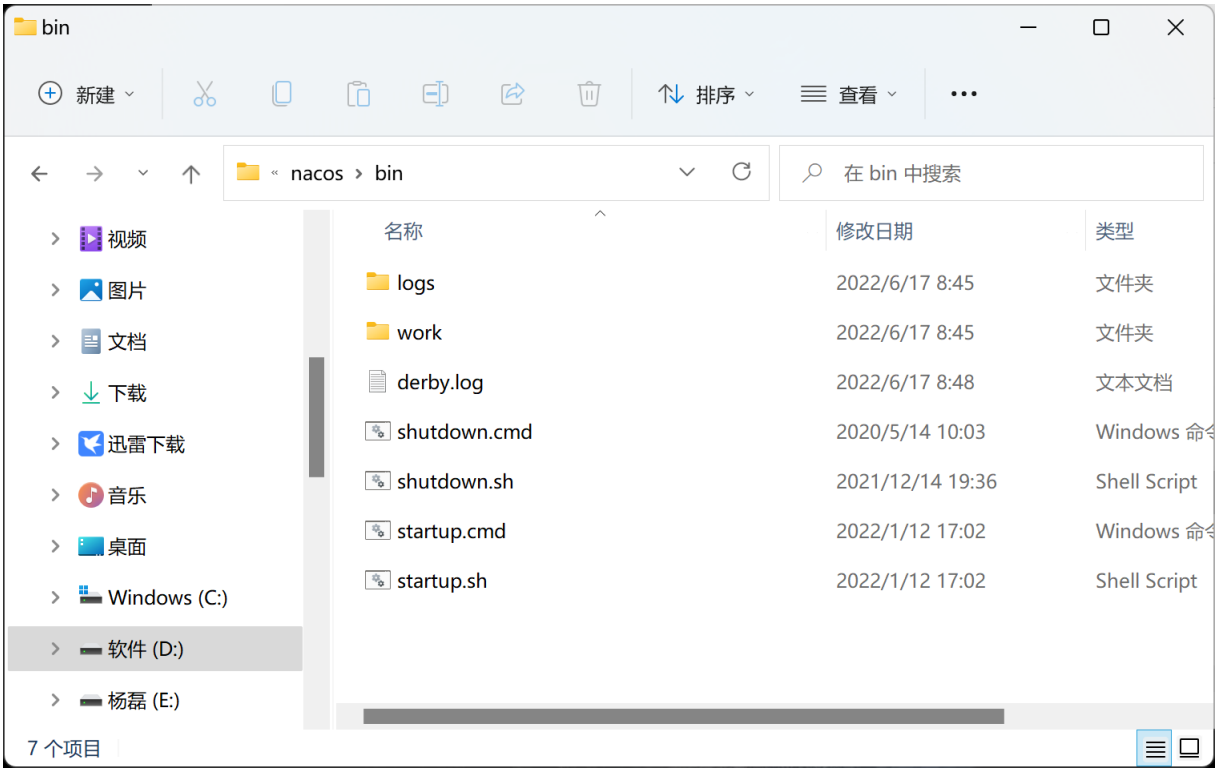
1.2 https://github.com/alibaba/nacos 如果打不开，可以使用 fastgithub。



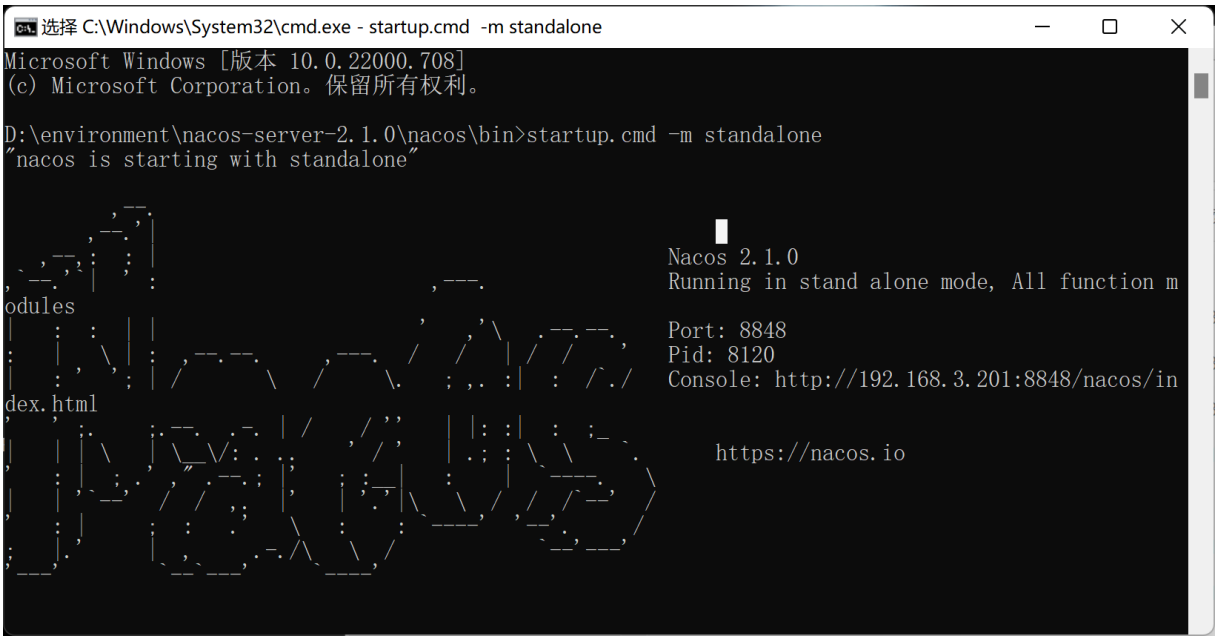

https://github.com/alibaba/nacos/releases/tag/2.1.0 点击 nacos-server-2.10.zip 下载。



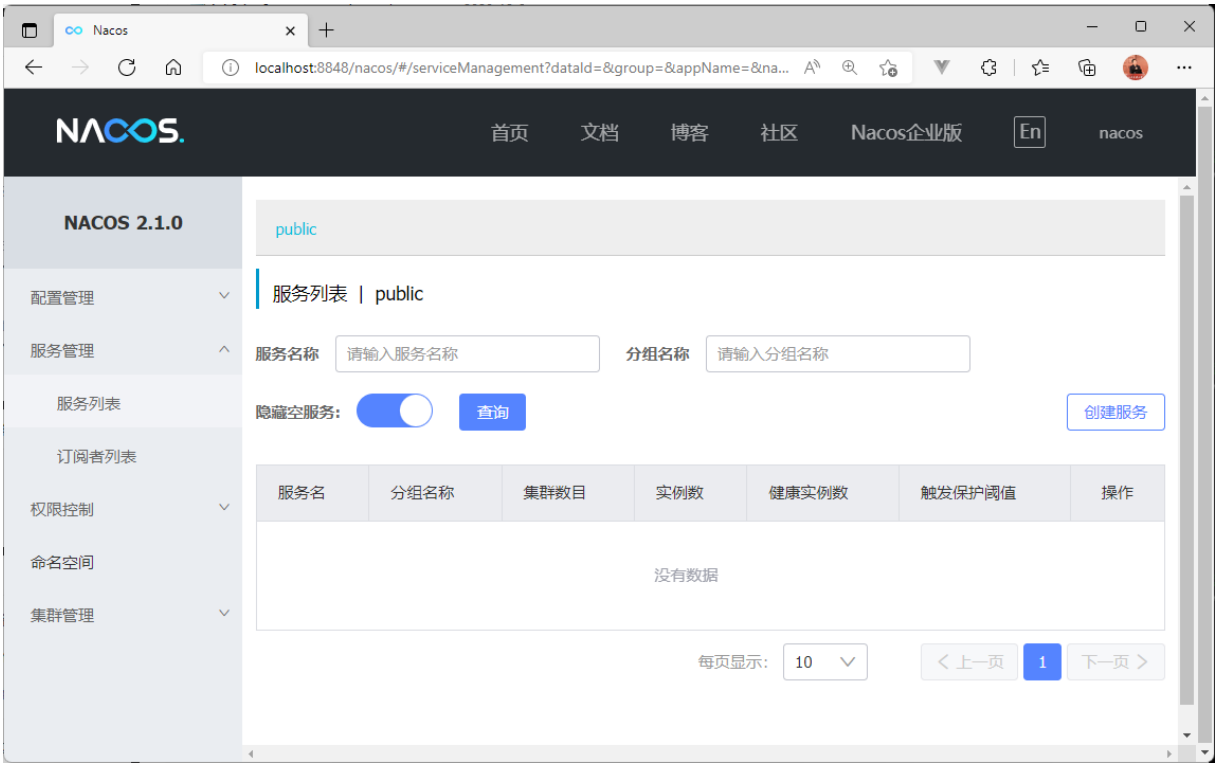

2、解压至 nacos 文件夹，进入 bin，在地址栏输入 cmd，打开命题行窗口。




3、在命令行窗口执行命令：startup.cmd -m standalone，以 standalone 模式运行。




4、打开浏览器，输入网址 http://localhost:8848/nacos，默认帐号和密码都是 nacos。




5、在前面项目的基础上，新建 Maven 子模块 ex05-nacos-8010。

【参考资料】[Spring Cloud Alibaba Reference Documentation]

6、修改 pom.xml，添加依赖

参考上面官方文档，添加 spring-cloud-starter-alibaba-nacos-discovery 依赖。

### 3.2. How to Introduce Nacos Discovery for service registration/discovery

please use the starter with the group ID as `com.alibaba.cloud` and the artifact ID as `spring-cloud-starter-alibaba-nacos-discovery`.

```xml
<dependency>
    <groupId>com.alibaba.cloud</groupId>
    <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
</dependency>
```

【参考资料】[https://mvnrepository.com/artifact/com.alibaba.cloud/spring-cloud-starter-alibaba-nacos-discovery/2021.1]

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>

    <dependency>
        <groupId>com.alibaba.cloud</groupId>
        <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
    </dependency>
</dependencies>
```

7、修改 application.yml，修改配置

```
server:
  port: 8010
spring:
  application:
    name: ex05-nacos-8010
  cloud:
    nacos:
      discovery:
        server-addr: 127.0.0.1:8848
management:
  endpoints:
    web:
      exposure:
        include: "*"
```

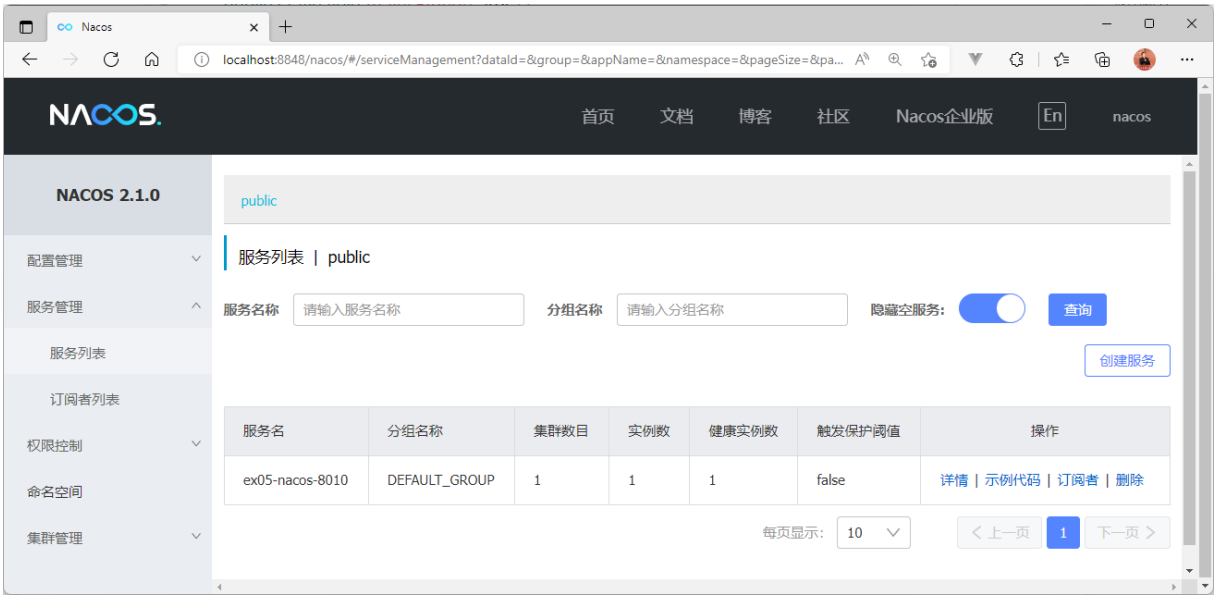
【参考资料】IDEA 中 application.yml 小绿叶加载不出来

https://blog.csdn.net/qq_40615403/article/details/118181413

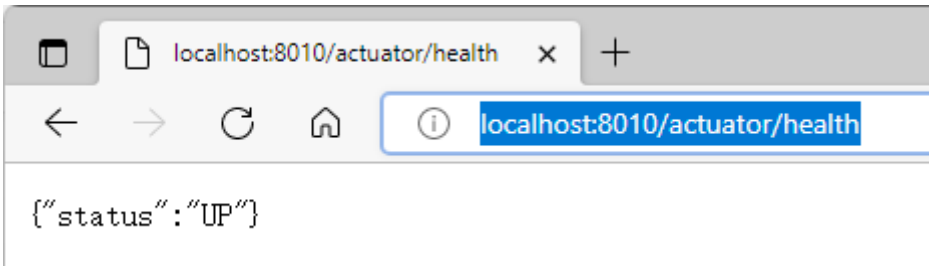
8、新建主程序 Ex05Application.java

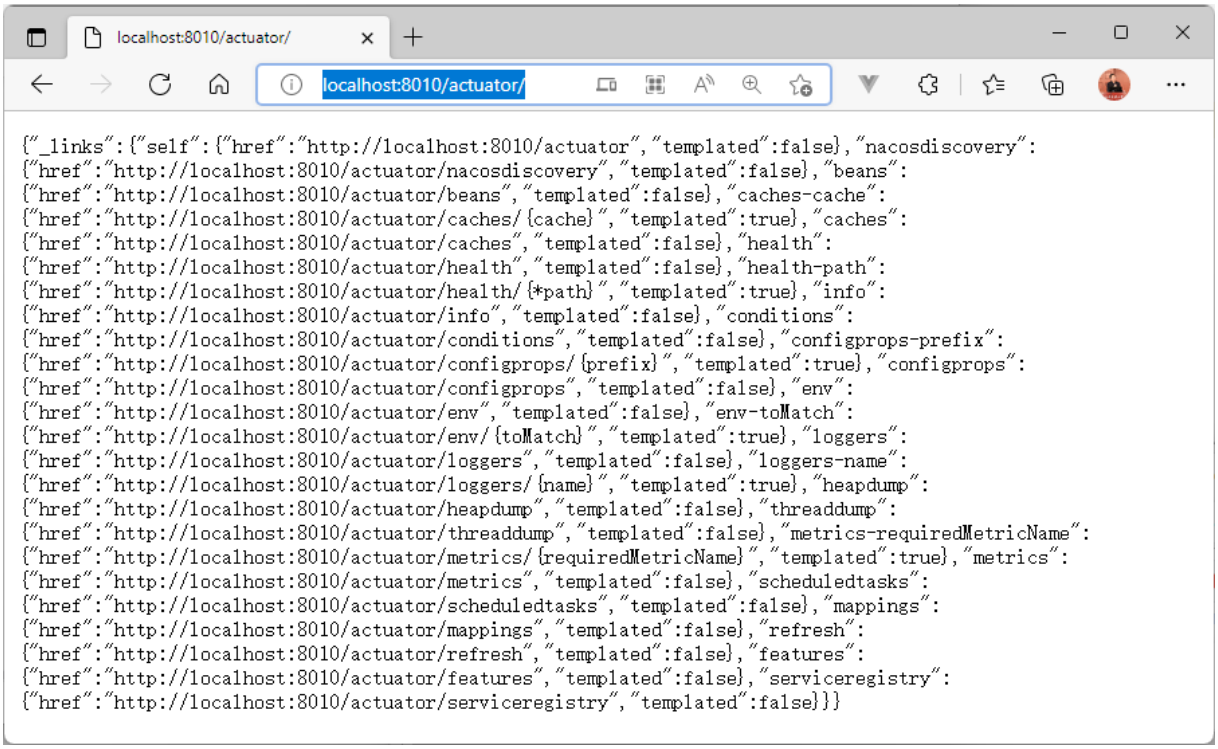
```
package cn.edu.sdjzu;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Ex05Application {
    public static void main(String[] args) {
        SpringApplication.run(Ex05Application.class, args);
    }
}
```

9、查看新注册的服务




查看服务健康情况：http://localhost:8010/actuator/health






{"_links":{"self":{"href":"http://localhost:8010/actuator","templated":false},"nacosdiscovery":
{"href":"http://localhost:8010/actuator/nacosdiscovery","templated":false},"beans":
{"href":"http://localhost:8010/actuator/beans","templated":false},"caches-cache":
{"href":"http://localhost:8010/actuator/caches/{cache}","templated":true},"caches":
{"href":"http://localhost:8010/actuator/caches","templated":false},"health":
{"href":"http://localhost:8010/actuator/health","templated":false},"health-path":
{"href":"http://localhost:8010/actuator/health/{*path}","templated":true},"info":
{"href":"http://localhost:8010/actuator/info","templated":false},"conditions":
{"href":"http://localhost:8010/actuator/conditions","templated":false},"configprops-prefix":
{"href":"http://localhost:8010/actuator/configprops/{prefix}","templated":true},"configprops":
{"href":"http://localhost:8010/actuator/configprops","templated":false},"env":
{"href":"http://localhost:8010/actuator/env","templated":false},"env-toMatch":
{"href":"http://localhost:8010/actuator/env/{toMatch}","templated":true},"loggers":
{"href":"http://localhost:8010/actuator/loggers","templated":false},"loggers-name":
{"href":"http://localhost:8010/actuator/loggers/{name}","templated":true},"heapdump":
{"href":"http://localhost:8010/actuator/heapdump","templated":false},"threaddump":
{"href":"http://localhost:8010/actuator/threaddump","templated":false},"metrics-requiredMetricName":
{"href":"http://localhost:8010/actuator/metrics/{requiredMetricName}","templated":true},"metrics":
{"href":"http://localhost:8010/actuator/metrics","templated":false},"scheduledtasks":
{"href":"http://localhost:8010/actuator/scheduledtasks","templated":false},"mappings":
{"href":"http://localhost:8010/actuator/mappings","templated":false},"refresh":
{"href":"http://localhost:8010/actuator/refresh","templated":false},"features":
{"href":"http://localhost:8010/actuator/features","templated":false},"serviceregistry":
{"href":"http://localhost:8010/actuator/serviceregistry","templated":false}}}

# 实验六 分布式配置

## 一、 目的与要求

掌握 Nacos 配置中心使用方法。

【参考资料】Nacos 配置中心官方文档，这个资料非常好，可以直接参考这个写教程。

## 二、 实验内容

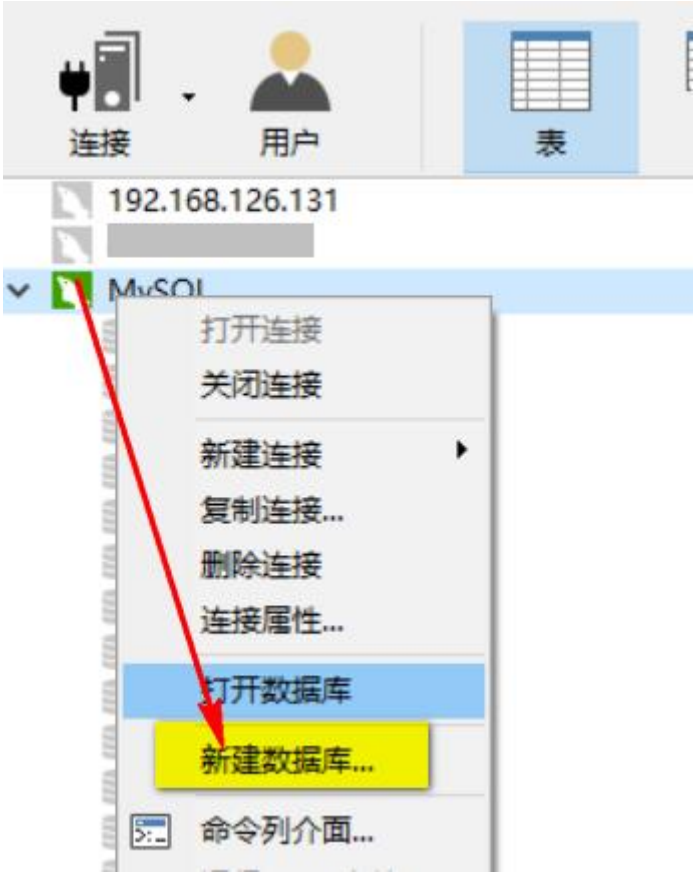
2.1 下载并运行 Nacos

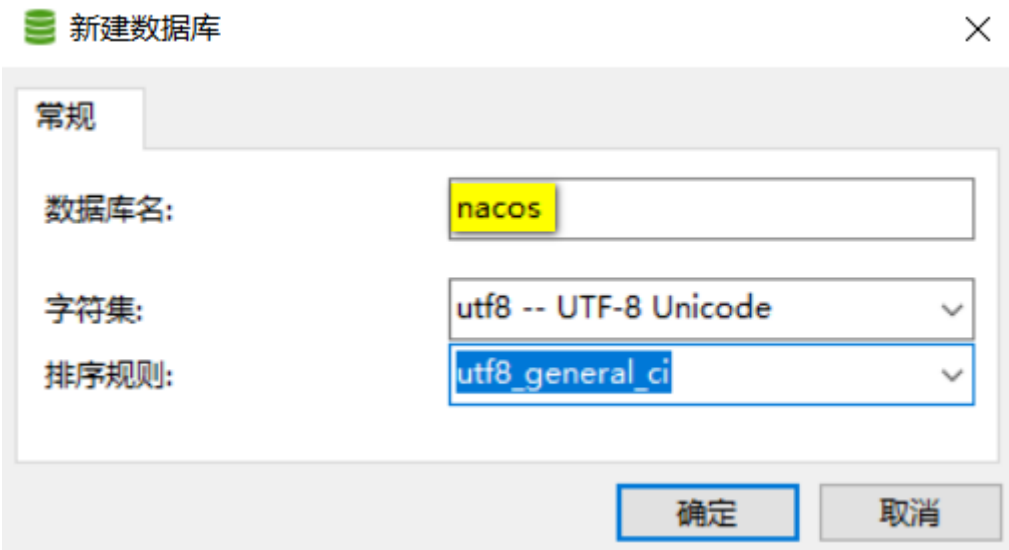
2.2 Nacos 配置中心持久化

2.3 编写微服务，将服务的配置信息设置到 Nacos 配置中心

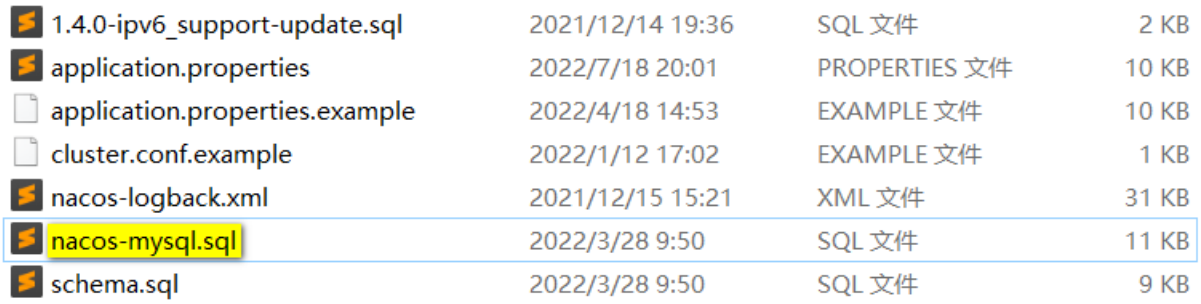
## 三、 实验指导

1、在 MySQL 中创建名称为 nacos 的数据库

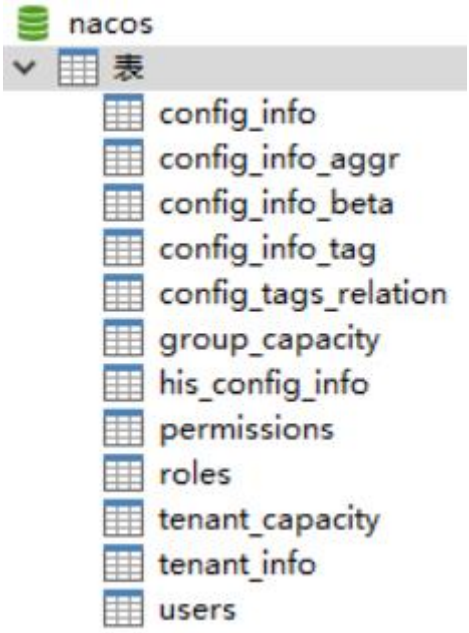





2、在 nacos 数据库中执行 nacos 根目录下 conf 目录下的 **nacos-mysql.sql** 数据库文件



| | | | |
|---|---|---|---|
| 1.4.0-ipv6_support-update.sql | 2021/12/14 19:36 | SQL 文件 | 2 KB |
| application.properties | 2022/7/18 20:01 | PROPERTIES 文件 | 10 KB |
| application.properties.example | 2022/4/18 14:53 | EXAMPLE 文件 | 10 KB |
| cluster.conf.example | 2022/1/12 17:02 | EXAMPLE 文件 | 1 KB |
| nacos-logback.xml | 2021/12/15 15:21 | XML 文件 | 31 KB |
| nacos-mysql.sql | 2022/3/28 9:50 | SQL 文件 | 11 KB |
| schema.sql | 2022/3/28 9:50 | SQL 文件 | 9 KB |




3、修改 nacos 根目录下 conf 目录下的 application.properties 文件

```
31  #*************** Config Module Related Configurations ***************
32  ### If use MySQL as datasource:
33  # spring.datasource.platform=mysql
34
35  ### Count of DB:
36  # db.num=1
37
38  ### Connect URL of DB:
39  # db.url.0=jdbc:mysql://127.0.0.1:3306/nacos?characterEncoding=utf8&conn
40  # db.user.0=nacos
41  # db.password.0=nacos
```

修改配置

```
1   #*************** Config Module Related Configurations ***************#
2   ### If use MySQL as datasource:
3   spring.datasource.platform=mysql
4
5   ### Count of DB:
6   db.num=1
7
8   ### Connect URL of DB:
9   db.url.0=jdbc:mysql://127.0.0.1:3306/nacos?characterEncoding=utf8&conne
10  db.user.0=改为你自己的数据库用户名
11  db.password.0=改为你的数据库密码
```

4、按照【实验五】中的方法启动 Nacos，在 nacos/bin 目录下执行 startup.cmd -m standalone

```
D:\nacos\bin>startup.cmd -m standalone
"nacos is starting with standalone"

Nacos 2.1.0
Running in stand alone mode, All function modules
Port: 8848
Pid: 137476
Console: http://192.168.1.93:8848/nacos/index.html

https://nacos.io
```

5、在前边项目基础上新建 ex06-nacos-config-8011 工程

6、在 ex06-nacos-config-8011 工程的 pom.xml 文件中增加如下依赖

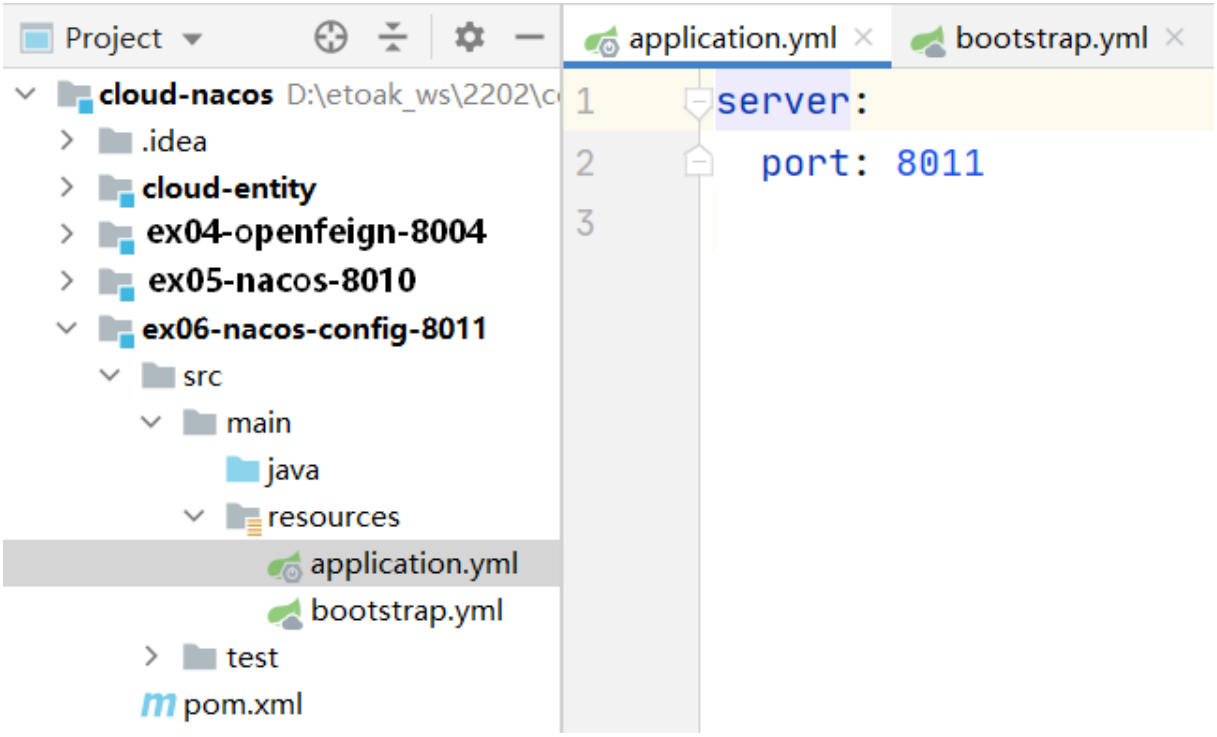
**【注意】**：使用配置中心必须引入 spring-cloud-starter-bootstrap 依赖，并在 src/main/resources 下创建 bootstrap.yml 文件

```
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Nacos 注册中心 -->
    <dependency>
      <groupId>com.alibaba.cloud</groupId>
      <artifactId>spring-cloud-starter-alibaba-nacos-
discovery</artifactId>
    </dependency>

    <!-- Nacos 配置中心 -->
    <dependency>
      <groupId>com.alibaba.cloud</groupId>
      <artifactId>spring-cloud-starter-alibaba-nacos-
config</artifactId>
    </dependency>

    <!-- spring-cloud-starter-bootstrap: bootstrap.yml -->
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-bootstrap</artifactId>
    </dependency>

    <!-- Spring Cloud 负载均衡组件 -->
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-loadbalancer</artifactId>
    </dependency>

    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
  </dependencies>
```

7、在 src/main/resources 下创建 application.yml 和 bootstrap.yml 文件

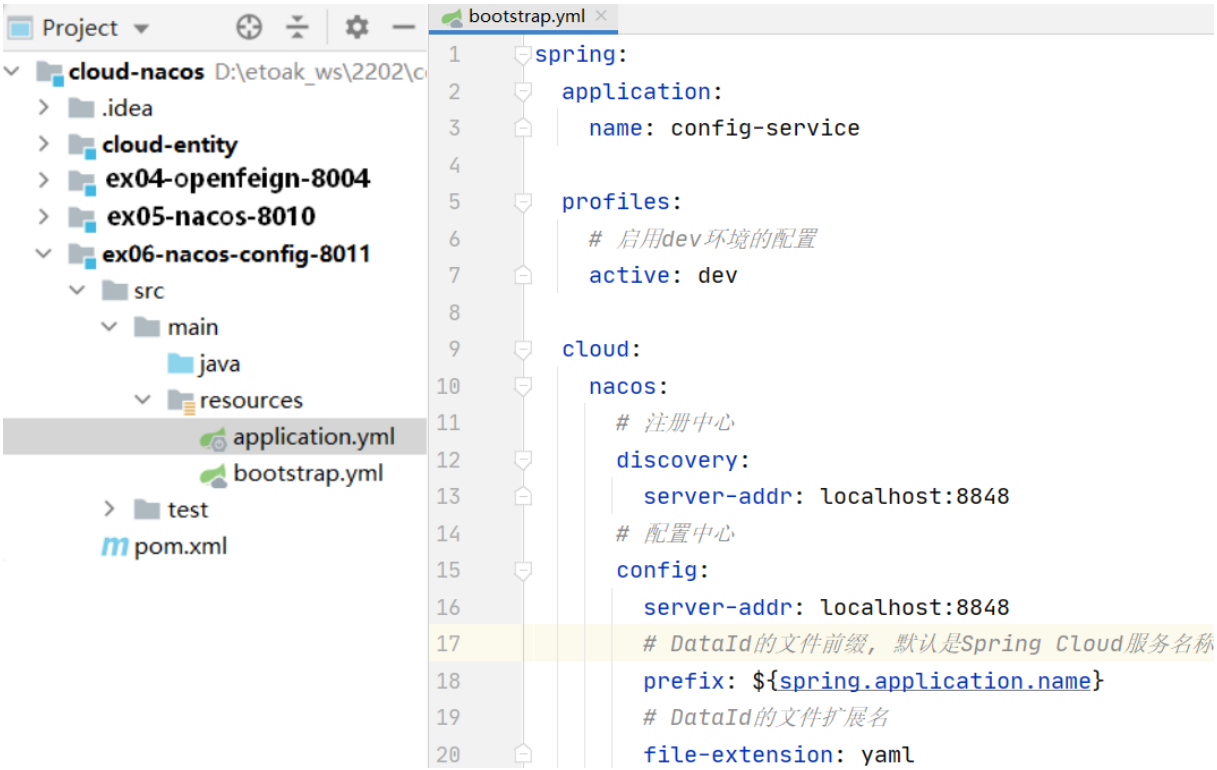
## application.yml 配置内容

```
server:
  port: 8011
```




## bootstrap.yml 配置内容

```yaml
spring:
  application:
    name: config-service

  profiles:
    # 启用 dev 环境的配置
    active: dev

  cloud:
    nacos:
      # 注册中心
      discovery:
        server-addr: localhost:8848
      # 配置中心
      config:
        server-addr: localhost:8848
        # DataId 的文件前缀，默认是 Spring Cloud 服务名称
        prefix: ${spring.application.name}
        # DataId 的文件扩展名
        file-extension: yaml
```




8、创建启动类

```
package cn.edu.sdjzu;

@SpringBootApplication
public class ConfigApp {

  public static void main(String[] args) {
    SpringApplication.run(ConfigApp.class, args);
  }
}
```

9、在 nacos 上配置 DataID

DataID 配置规则

${spring.cloud.nacos.config.prefix}-${spring.profiles.active}.${spring.cloud.nacos.config.file-extension}

**${spring.cloud.nacos.config.prefix}**：默认值是服务名 spring.application.name

**${spring.profiles.active}**：不写的话，默认是空值

**${spring.cloud.nacos.config.file-extension}**：文件扩展名

10、创建 StudentController，获取配置中心配置项

```
package cn.edu.sdjzu.controller;

import org.springframework.beans.factory.annotation.Value;
import
org.springframework.cloud.context.config.annotation.RefreshScope;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/student")
@RefreshScope // 当配置中心的配置修改了，自动刷新配置
public class StudentController {

  /** 获取配置中心的配置项 */
  @Value("${student.id:100}")
  int id;

  /** 获取配置中心的配置项 */
  @Value("${student.name:sdjzu}")
  String name;

  /**
   * 测试从配置中心获取配置
   */
  @RequestMapping("/config")
  public Object config() {
    Map<String, Object> resultMap = new HashMap<>();
    resultMap.put("id", this.id);
    resultMap.put("name", this.name);
    return resultMap;
  }
}
```

11、 在浏览器中进行测试

localhost:8011/student/config

应用 书签 ETOAK-API tsinghua镜像站 阿里云-容器镜像服务


```
{
    name: "zhangsan",
    id: 1
}
```

# 实验七 服务容错（限流、熔断降级）

## 一、 目的与要求

理解服务熔断技术

掌握 Sentinel 组件的使用以及使用 Sentinel 实现服务的限流与熔断

## 二、 实验内容

2.1 下载、安装、运行 Sentinel

2.2 使用 Sentinel 保护微服务

## 三、 实验指导

**1. Sentinel 介绍**

1.1. Sentinel 官网

1.2. Sentinel 简介

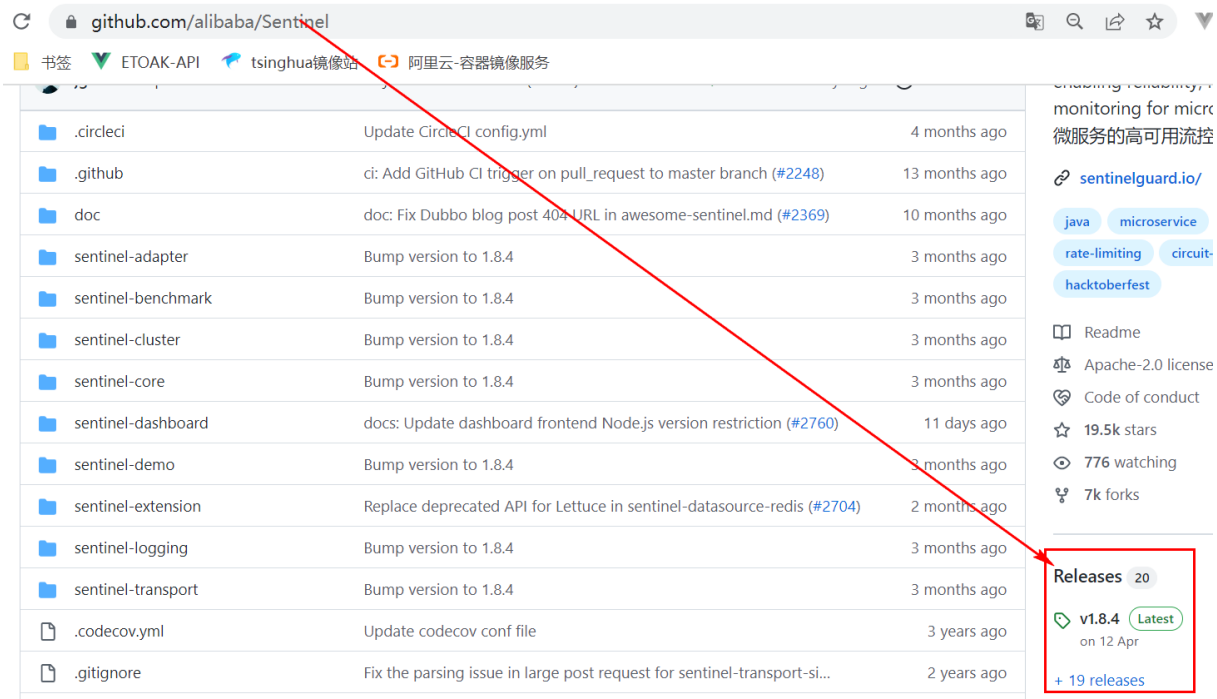
Sentinel 是面向分布式、多语言异构化服务架构的流量治理组件，主要以流量为切入点，从流量控制、流量路由、熔断降级、系统自适应保护等多个维度来帮助用户保障微服务的稳定性。

1.3. Sentinel 特征

- **丰富的应用场景**：Sentinel 承接了阿里巴巴近 10 年的双十一大促流量的核心场景，例如秒杀（即突发流量控制在系统容量可以承受的范围）、消息削峰填谷、集群流量控制、实时熔断下游不可用应用等。
- **完备的实时监控**：Sentinel 同时提供实时的监控功能。您可以在控制台中看到接入应用的单台机器秒级数据，甚至 500 台以下规模的集群的汇总运行情况。
- **广泛的开源生态**：Sentinel 提供开箱即用的与其它开源框架/库的整合模块，例如与 Spring Cloud、Apache Dubbo、gRPC、Quarkus 的整合。您只需要引入相应的依赖并进行简单的配置即可快速地接入 Sentinel。同时 Sentinel 提供 Java/Go/C++ 等多语言的原生实现。
- **完善的 SPI 扩展机制**：Sentinel 提供简单易用、完善的 SPI 扩展接口。您可以通过实现扩展接口来快速地定制逻辑。例如定制规则管理、适配动态数据源等。

**2. 下载 Sentinel**

2.1. 下载地址：https://github.com/alibaba/Sentinel/releases



## 3. 运行 Sentinel

3.1. 在 Sentinel 目录中执行：java -jar sentinel

```
D:\etoak_ws\soft\7.Cloud_Alibaba>java -jar sentinel-dashboard-1.8.2.jar
INFO: Sentinel log output type is: file
INFO: Sentinel log charset is: utf-8
INFO: Sentinel log base directory is: C:\Users\etoak\logs\csp\
INFO: Sentinel log name use pid is: false

 :: Spring Boot ::        (v2.0.5.RELEASE)
```

3.2. 浏览器访问：localhost:8080

3.3. 输入用户名和密码：sentinel/sentinel

4. 创建子模块 ex07-sentinel-8012，在 pom.xml 中引入 sentinel 依赖

```
    <dependencies>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
      </dependency>

      <dependency>
        <groupId>com.alibaba.cloud</groupId>
        <artifactId>spring-cloud-starter-alibaba-nacos-
discovery</artifactId>
      </dependency>

      <dependency>
        <groupId>com.alibaba.cloud</groupId>
        <artifactId>spring-cloud-starter-alibaba-
sentinel</artifactId>
      </dependency>
    </dependencies>
```

5. 在 application.yml 中配置 sentinel 控制台地址

```
server:
  port: 8012

spring:
  application:
    name: sentinel-service

  cloud:
    nacos:
      discovery:
        server-addr: localhost:8848

    sentinel:
      transport:
        dashboard: localhost:8080
```

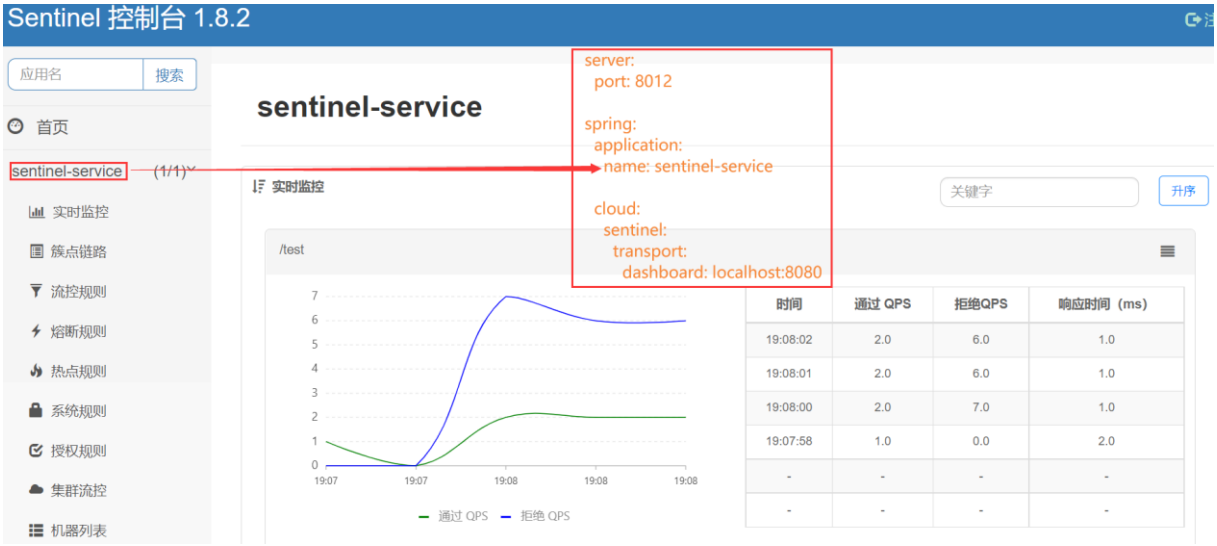
6. 编写启动类：SentinelApp

```
package cn.edu.sdjzu;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SentinelApp {

  public static void main(String[] args) {
    SpringApplication.run(SentinelApp.class, args);
  }
}
```

7. 编写测试 Controller

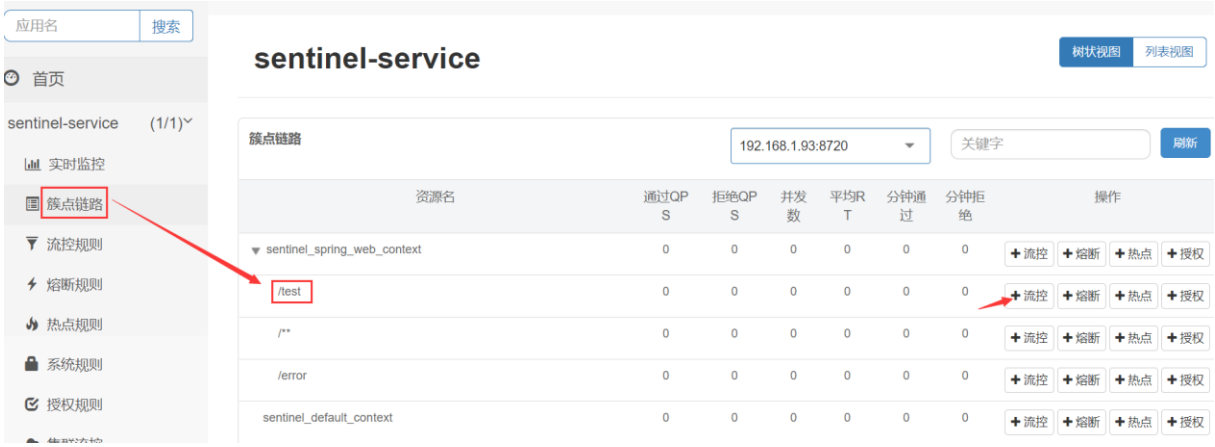
```
package cn.edu.sdjzu.controller;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

  @RequestMapping("/test")
  public String test() {
    return "success! ";
  }

}
```

8. 访问测试类中的接口（localhost:8012/test），触发 Sentinel 监控

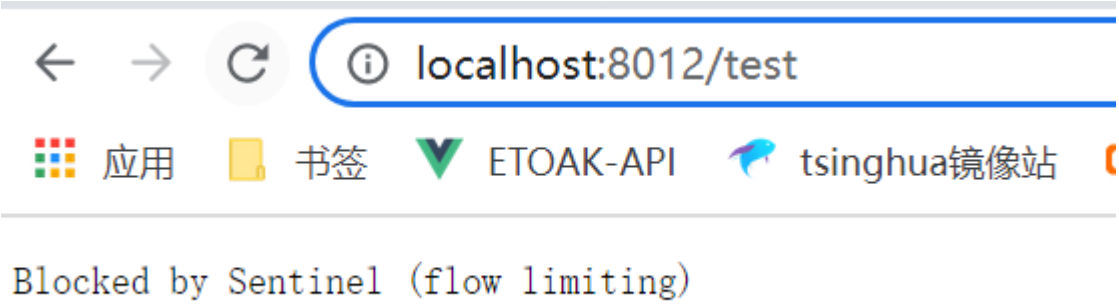


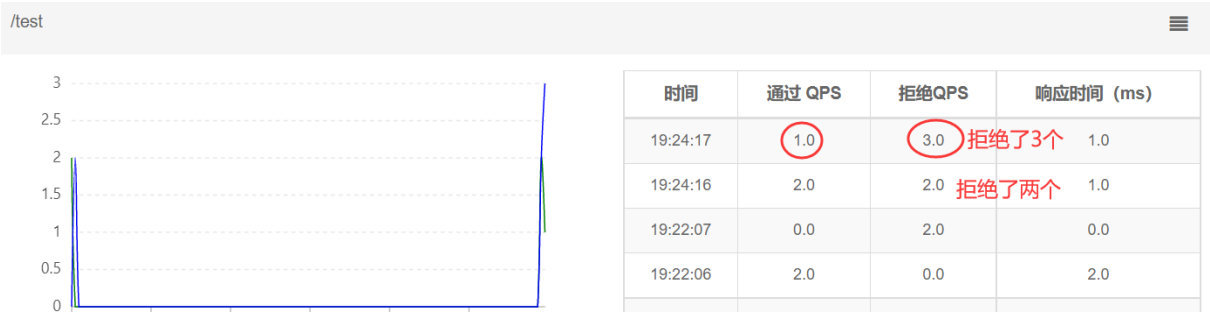

9. 测试直接流控模式

9.1. 点击簇点链路菜单，点击/test 接口的右侧的**流控**按钮，创建流控规则（设置每秒只能并发两个请求）



9.2. 在浏览器上进行测试，发现被流控了




10. 测试关联流控模式

10.1. 什么是关联流控：

当两个资源之间具有资源争抢或者依赖关系的时候，这两个资源便具有了关联。比如对数据库同一个字段的读操作和写操作存在争抢，读的速度过高会影响写得速度，写的速度过高会影响读的速度。如果放任读写操作争抢资源，则争抢本身带来

的开销会降低整体的吞吐量。可使用关联限流来避免具有关联关系的资源之间过度的争抢。

简单来说就是如果与 a 关联的 b 触发了限流阈值，则对 a 进行限流

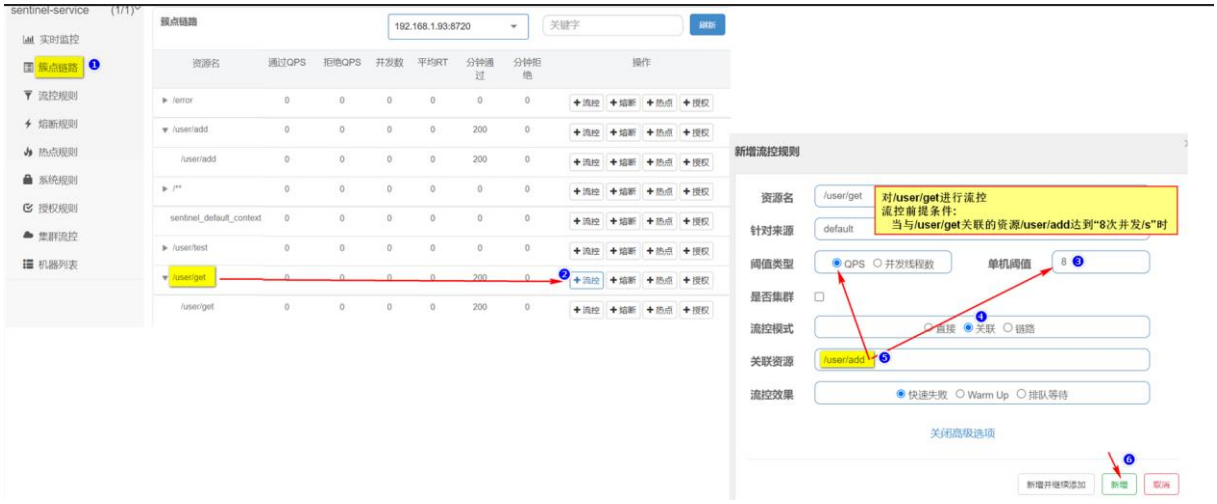
10.2. 在 UserController 增加两个测试接口，添加（/user/add）和查询(/user/get)

```
@RestController
@RequestMapping("/user")
public class UserController {

  @RequestMapping(value = "/add", produces =
"text/plain;charset=utf-8")
  public String add() {
    return "添加用户，写数据库";
  }

  @RequestMapping(value = "/get", produces =
"text/plain;charset=utf-8")
  public String get() {
    return "查询用户，读数据库";
  }
}
```

10.3. 在 application.yml 增加配置

```
spring:
  cloud:
    sentinel:
      web-context-unify: false # 将每个服务分开，不从根链路触发
```

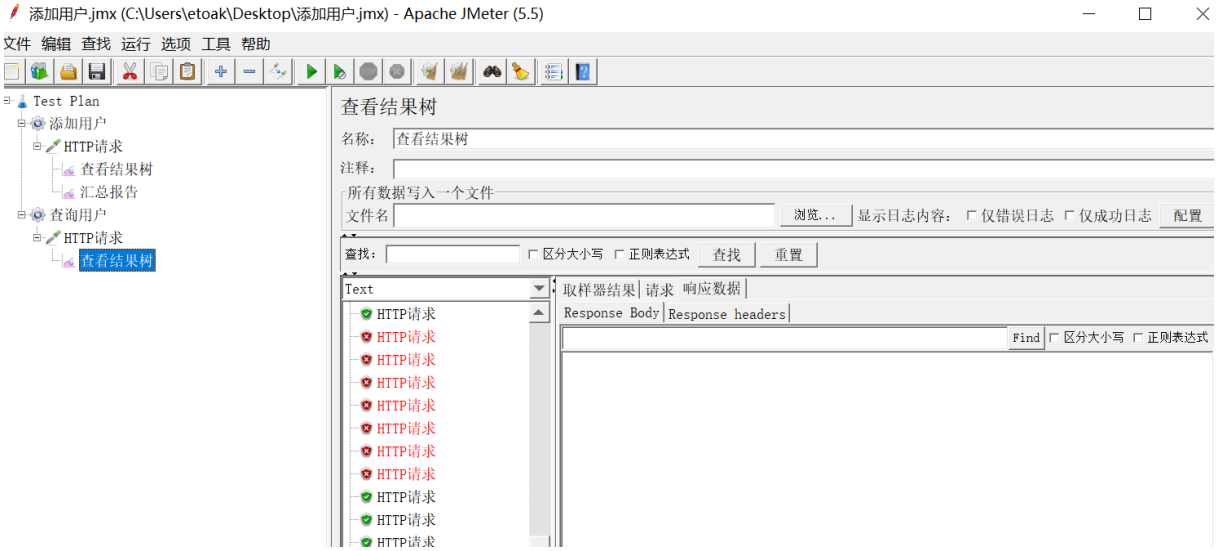
10.4. 在 sentinel 控制设置流控规则




## 10.5. 使用 JMeter 进行测试

创建两个线程组，添加 HTTP 请求，为 HTTP 请求添加"监听器->查看结果树"

## 10.6. 控制台流控结果


添加用户.jmx (C:\Users\etoak\Desktop\添加用户.jmx) - Apache JMeter (5.5)
文件 编辑 查找 运行 选项 工具 帮助
Test Plan
添加用户
HTTP请求
查看结果树
汇总报告
查询用户
HTTP请求
查看结果树
查看结果树
名称: 查看结果树
注释:
所有数据写入一个文件
文件名
浏览...
显示日志内容: 仅错误日志 仅成功日志
配置
查找:
区分大小写 正则表达式
查找
重置
Text
HTTP请求
取样器结果 请求 响应数据
Response Body Response headers
Find 区分大小写 正则表达式

# 实验八 服务网关

## 一、 目的与要求

理解服务网关

服务网关的作用

掌握服务网关的使用

掌握 Spring Cloud Gateway 的核心概念

- **Route**：由 ID、URI、一组 Predicates 和一组过滤器组成。
- **Predicate**：JDK 8 中提供的函数式接口，可以对 HTTP 请求中的任何属性进行断言
- **Filter**：过滤器，可以在请求和响应前后对 HTTP 请求进行拦截处理

## 二、 实验内容

2.1 使用 Spring Cloud Gateway 作为代理服务器，将请求代理到山东建筑大学官方网站

2.2 使用 Spring Cloud Gateway 对注册中心的服务进行路由

## 三、 实验指导

1. 使用 Spring Cloud Gateway 作为代理服务器，将请求代理到山东建筑大学官方网站

   1.1. 在前边项目基础上，新建 **ex08-gateway-8013** 工程

   1.2. 在 **ex08-gateway-8013** 工程的 pom.xml 文件中增加如下依赖

```
<dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-gateway</artifactId>
    </dependency>
</dependencies>
```

   1.3. 在 src/main/resources 下创建 application.yml，并在文件添加如下配置

```
server:
  port: 8013

# Route(id, uri, 一组断言、一组过滤器组成)
spring:
  cloud:
    gateway:
      routes:
        - id: et2202
          uri: https://www.sdjzu.edu.cn/
          predicates:
            # 访问 localhost:8013 时 路由到
https://www.sdjzu.edu.cn/
            - Path=/**
```

1.4. 编写启动类 cn.edu.sdjzu.GatewayApp

```
package cn.edu.sdjzu;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GatewayApp {

  public static void main(String[] args) {
    SpringApplication.run(GatewayApp.class);
  }
}
```

1.5. 在浏览器中输入：http://localhost:8013 进行测试

## 2. 使用 Spring Cloud Gateway 对 Nacos 注册中心的服务进行路由

2.1. 在前边项目基础上，新建 ex09-gateway-8014 工程

2.2. 在 ex09-gateway-8014 工程的 pom.xml 文件中增加如下依赖

```
<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
  </dependency>

  <dependency>
    <groupId>com.alibaba.cloud</groupId>
    <artifactId>spring-cloud-starter-alibaba-nacos-
discovery</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-loadbalancer</artifactId>
  </dependency>
</dependencies>
```

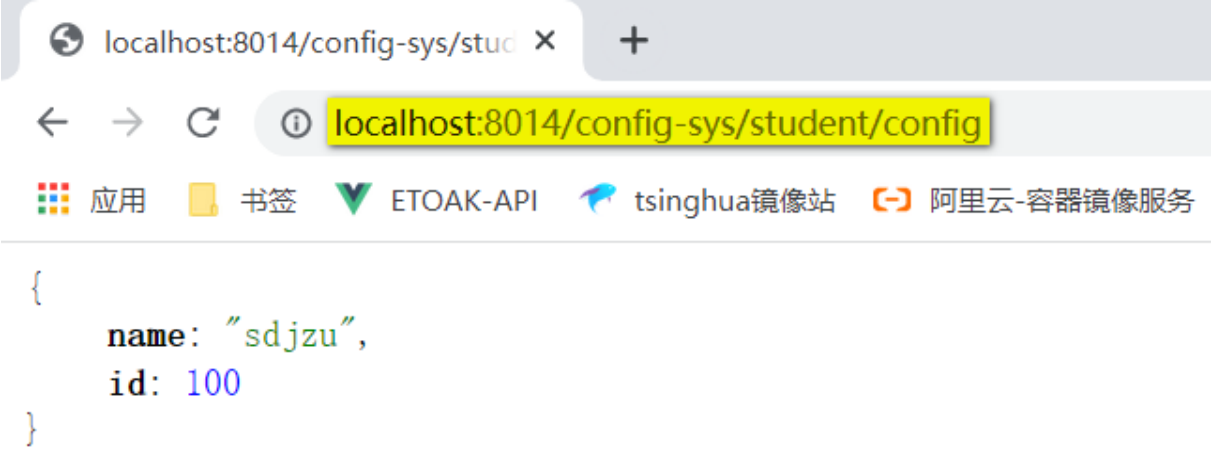
2.3. 在 src/main/resources 下创建 application.yml，并在文件添加如下配置

```yaml
server:
  port: 8014

spring:
  application:
    name: gateway

  cloud:
    nacos:
      # Nacos 注册中心
      discovery:
        server-addr: localhost:8848

    gateway:
      discovery:
        # 可以根据服务名称路由服务
        locator:
          enabled: true
      routes:
        - id: config-service
          uri: lb://config-service
          predicates:
            # 访问 http://localhost:8014/config-sys/student/config
            # 路由到 lb://config-service/student/config
            - Path=/config-sys/**
          filters:
            #地址路由转发前跳过第一层的请求，也就是去掉/config-sys
            - StripPrefix=1
```

### 2.4. 编写启动类 cn.edu.sdjzu.GatewayApp

```
package cn.edu.sdjzu;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class GatewayApp {

  public static void main(String[] args) {
    SpringApplication.run(GatewayApp.class);
  }
}
```

2.5. 在浏览器输入：http://localhost:8014/config-sys/student/config



# 实验九 分布式事务 Seata

## 一、 目的与要求

了解什么是事务、事务的特征、事务的分类；

单机模式下的事务和微服务模式下的事务；

学会 Seata 的使用。

## 二、 实验内容

2.1 什么是事务

2.2 事务的特征

2.3 单机模式下的事务与微服务模式下的事务之间的区别

2.4 如何在 Spring Cloud Alibaba 框架中实现事务

2.5 Seata 的使用

## 三、 实验指导

### 黑马 Seata 入门到实战教程

https://www.bilibili.com/video/BV16h4y1G7kd/?spm_id_from=333.337.search-card.all.click&vd_source=c5e65f020919c07bb4d1cc902ee3c1d7

### 3.1 事务简介

事务(Transaction)是访问并可能更新数据库中各种数据项的一个程序执行单元(unit)。在关系数据库中，一个事务由一组 SQL 语句组成。事务应该具有 4 个属性：原子性、一致性、隔离性、持久性。这四个属性通常称为 ACID 特性。

**原子性**（atomicity）：事务是一个不可分割的工作单位，事务中包括的诸操作要么都做，要么都不做。

**一致性**（consistency）：事务必须是使数据库从一个一致性状态变到另一个一致性状态，事务的中间状态不能被观察到的。

**隔离性**（isolation）：一个事务的执行不能被其他事务干扰。即一个事务内部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。隔离性又分为四个级别：读未提交(read uncommitted)、读已提交(read committed，解决脏读)、可重复读

(repeatable read，解决虚读)、串行化(serializable，解决幻读)。

**持久性**（durability）：持久性也称永久性（permanence），指一个事务一旦提交，它对数据库中数据的改变就应该是永久性的。接下来的其他操作或故障不应该对其有任何影响。

任何事务机制在实现时，都应该考虑事务的 ACID 特性，包括：本地事务、分布式事务，及时不能都很好的满足，也要考虑支持到什么程度。

## 3.2 本地事务

大多数场景下，我们的应用都只需要操作单一的数据库，这种情况下的事务称之为本地事务(Local Transaction)。本地事务的 ACID 特性是数据库直接提供支持。在 JDBC 编程中，我们通过 java.sql.Connection 对象来开启、关闭或者提交事务。代码如下所示：

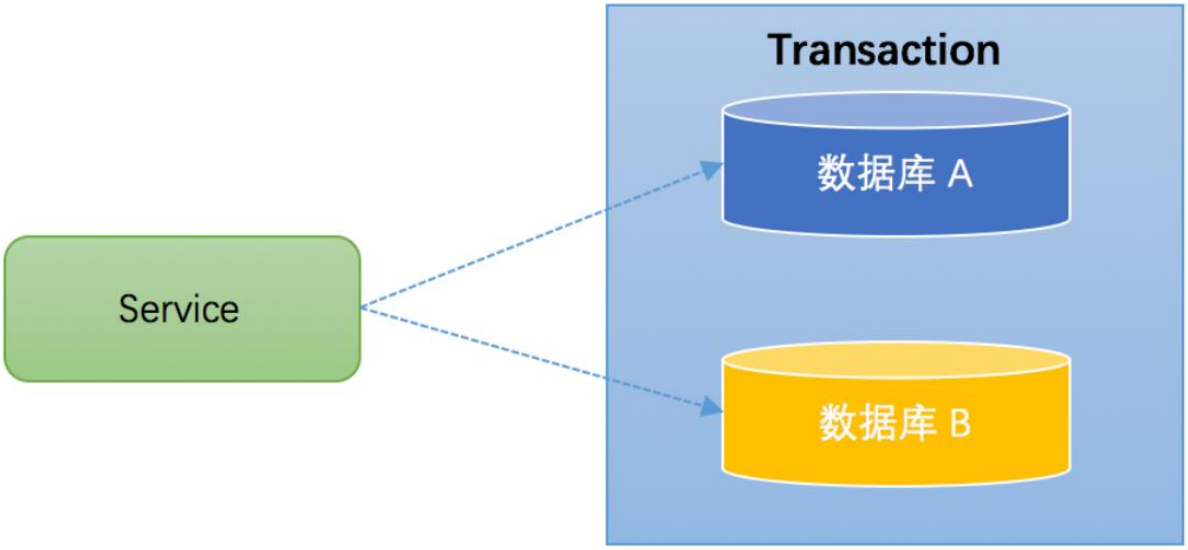
```
Connection conn = ... //获取数据库连接
conn.setAutoCommit(false); //开启事务
try{
    //...执行增删改查 sql
    conn.commit(); //提交事务
 }catch (Exception e) {
   conn.rollback();//事务回滚
 }finally{
    conn.close();//关闭链接
 }
```

## 3.3 分布式事务

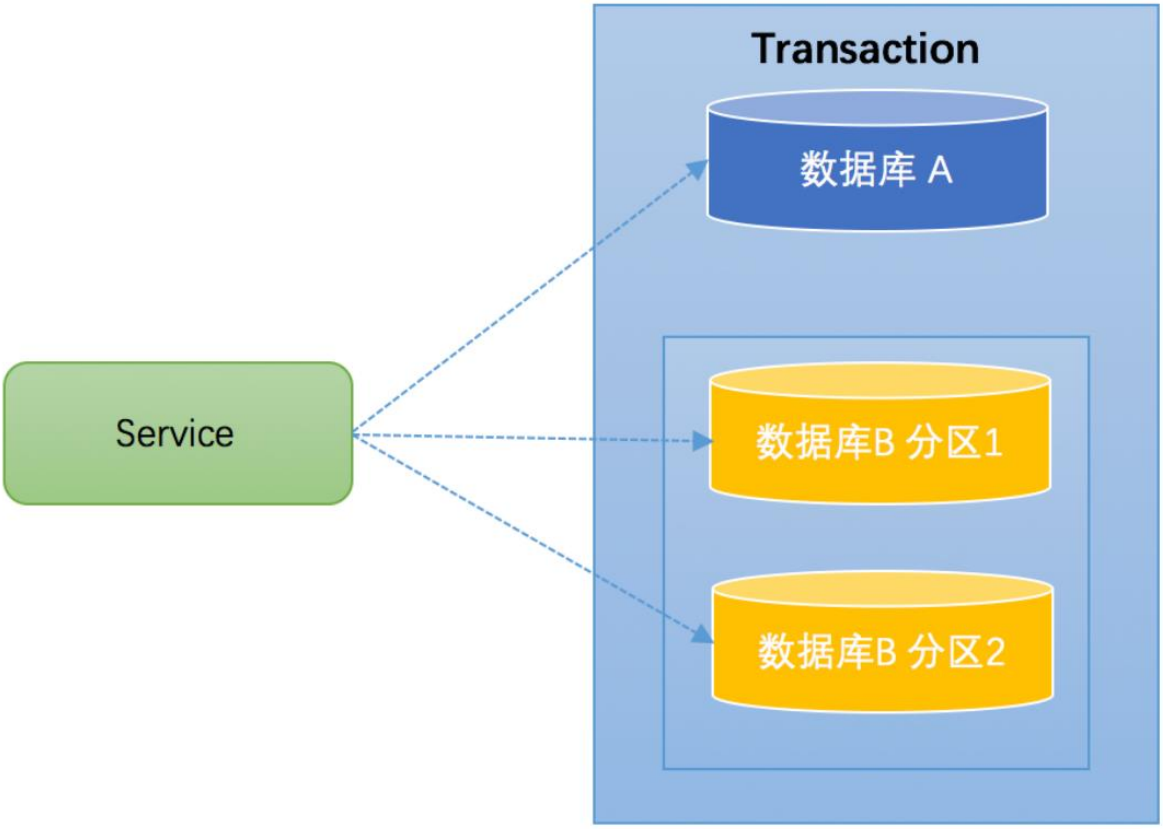
当下互联网发展如火如荼，绝大部分公司都进行了数据库拆分和服务化(SOA)。在这种情况下，完成某一个业务功能可能需要横跨多个服务，操作多个数据库。这就涉及到到了分布式事务，用需要操作的资源位于多个资源服务器上，而应用需要保证对于多个资源服务器的数据的操作，要么全部成功，要么全部失败。本质上来说，分布式事务就是为了保证不同资源服务器的数据一致性。

典型的分布式事务场景：

（1）跨库事务指的是，一个应用某个功能需要操作多个库，不同的库中存储不同的业务数据。下图演示了一个服务同时操作 2 个库的情况。



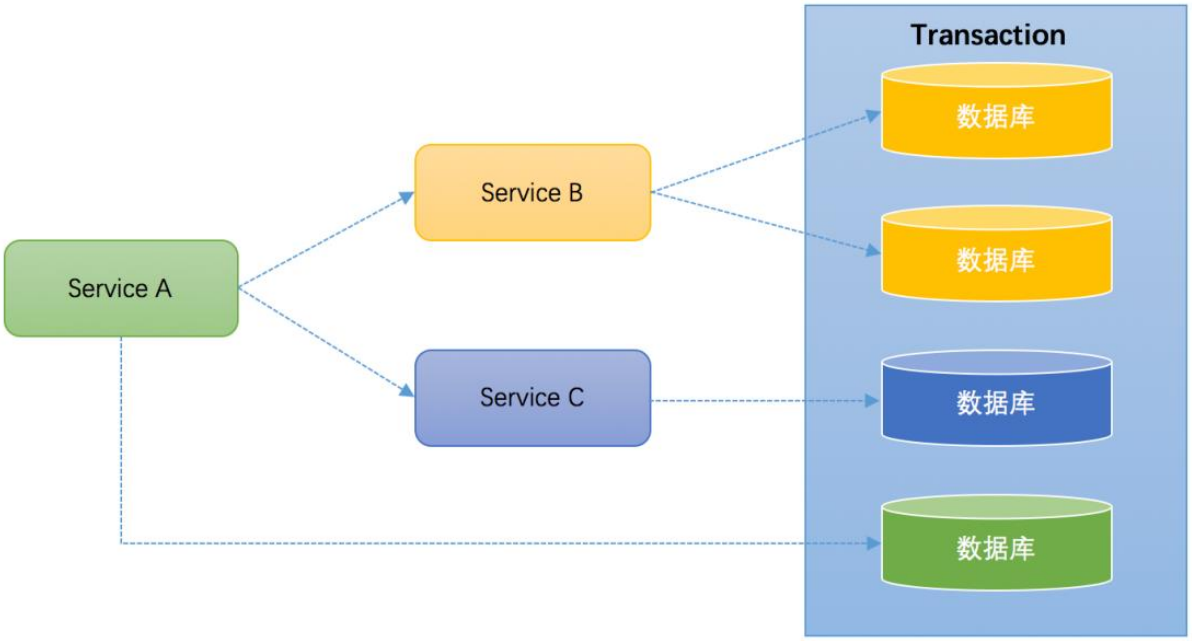

（2）分库分表。通常一个库数据量比较大或者预期未来的数据量比较大，都会进行水平拆分，也就是分库分表。如下图，将数据库 B 拆分成了 2 个库：




对于分库分表的情况，一般开发人员都会使用一些数据库中间件来降低 sql 操作的复杂性。如，对于 sql：insert into user(id,name) values (1,"张三"),(2,"李四")。这条 sql 是操作单库的语法，单库情况下，可以保证事务的一致性。但是由于现在进行了分库分表，开发人员希望将 1 号记录插入分库 1，2 号记录插入分库 2。所以数据库中间件要将其改写为 2 条 sql，分别插入两个

不同的分库，此时要保证两个库要不都成功，要不都失败，因此基本上所有的数据库中间件都面临着分布式事务的问题。

（3）服务化。微服务架构是目前一个比较一个比较火的概念。例如上面提到的一个案例，某个应用同时操作了多个库，这样的应用业务逻辑必然非常复杂，对于开发人员是极大的挑战，应该拆分成不同的独立服务，以简化业务逻辑。拆分后，独立服务之间通过 RPC 框架来进行远程调用，实现彼此的通信。下图演示了一个 3 个服务之间彼此调用的架构：




Service A 完成某个功能需要直接操作数据库，同时需要调用 Service B 和 Service C，而 Service B 又同时操作了 2 个数据库，Service C 也操作了一个库。需要保证这些跨服务的对多个数据库的操作要不都成功，要不都失败，实际上这可能是最典型的分布式事务场景。

小结：上述讨论的分布式事务场景中，无一例外的都直接或者间接的操作了多个数据库。如何保证事务的 ACID 特性，对于分布式事务实现方案而言，是非常大的挑战。同时，分布式事务实现方案还必须要考虑性能的问题，如果为了严格保证 ACID 特性，导致性能严重下降，那么对于一些要求快速响应的业务，是无法接受的。

## 3.4 常见的分布式事务解决方案

（1）seata 阿里分布式事务框架

（2）消息队列

（3）saga

（4）XA

他们有一个共同点，都是“两阶段(2PC)”。“两阶段”是指完成整个分布式事务，划分成两个步骤完成。

2PC，也就是两阶段提交协议，顾名思义，分为两个阶段：Prepare 和 Commit

## 3.5 Seata 的使用

（1）搭建 TC（Server 端）

官网：https://seata.io/zh-cn/index.html

源码: https://github.com/seata/seata

官方 Demo: https://github.com/seata/seata-samples

| Spring Cloud Alibaba Version | Sentinel Version | Nacos Version | RocketMQ Version | Dubbo Version | Seata Version |
|---|---|---|---|---|---|
| 2.2.10-RC1 | 1.8.6 | 2.2.0 | 4.9.4 | ~ | 1.6.1 |
| 2022.0.0.0-RC1 | 1.8.6 | 2.2.1-RC | 4.9.4 | ~ | 1.6.1 |
| 2.2.9.RELEASE | 1.8.5 | 2.1.0 | 4.9.4 | ~ | 1.5.2 |
| 2021.0.4.0 | 1.8.5 | 2.0.4 | 4.9.4 | ~ | 1.5.2 |
| 2.2.8.RELEASE | 1.8.4 | 2.1.0 | 4.9.3 | ~ | 1.5.1 |
| 2021.0.1.0 | 1.8.3 | 1.4.2 | 4.9.2 | ~ | 1.4.2 |
| 2.2.7.RELEASE | 1.8.1 | 2.0.3 | 4.6.1 | 2.7.13 | 1.3.0 |
| 2.2.6.RELEASE | 1.8.1 | 1.4.2 | 4.4.0 | 2.7.8 | 1.3.0 |
| 2021.1 or 2.2.5.RELEASE or 2.1.4.RELEASE or 2.0.4.RELEASE | 1.8.0 | 1.4.1 | 4.4.0 | 2.7.8 | 1.3.0 |
| 2.2.3.RELEASE or 2.1.3.RELEASE or 2.0.3.RELEASE | 1.8.0 | 1.3.3 | 4.4.0 | 2.7.8 | 1.3.0 |
| 2.2.1.RELEASE or 2.1.2.RELEASE or 2.0.2.RELEASE | 1.7.1 | 1.2.1 | 4.4.0 | 2.7.6 | 1.2.0 |

根据版本的对应关系，我们去官网下载 Seata1.4.2 版本的服务端。

Server 端的存储模式：

1.file：(默认)单机模式，全局事务会话信息内存中读写并持久化本地文件 root.data，性能较高(默认）

2.db：（5.7+）高可用模式，全局事务会话信息通过 db 共享，相应性能差些

3.redis：Seata-Server 1.3 及以上版本支持,性能较高,存在事务信息丢失风险,请提前配置适合当前场景的 redis 持久化配置

我们采用第二种 DB 的方式来部署 Server 端，按照如下步骤修改配置文件：

- 打开 config/file.conf
- 修改 mode="db"
- 修改数据库连接信息（URL\USERNAME\PASSWORD) 数据库要 5.7 以上的版本
- 创建数据库 seata_server
- 新建表： 可以去 seata 提供的资源信息中下载：

https://github.com/seata/seata/blob/2.x/script/server/db/mysql.sql

```
-- ------------------------------- The script used when
storeMode is 'db' --------------------------------
-- the table to store GlobalSession data
CREATE TABLE IF NOT EXISTS `global_table`
(
    `xid`                       VARCHAR(128) NOT NULL,
    `transaction_id`            BIGINT,
    `status`                    TINYINT      NOT NULL,
    `application_id`            VARCHAR(32),
    `transaction_service_group` VARCHAR(32),
    `transaction_name`          VARCHAR(128),
    `timeout`                   INT,
    `begin_time`                BIGINT,
    `application_data`          VARCHAR(2000),
    `gmt_create`                DATETIME,
    `gmt_modified`              DATETIME,
    PRIMARY KEY (`xid`),
    KEY `idx_status_gmt_modified` (`status` , `gmt_modified`),
    KEY `idx_transaction_id` (`transaction_id`)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4;

-- the table to store BranchSession data
CREATE TABLE IF NOT EXISTS `branch_table`
(
    `branch_id`         BIGINT       NOT NULL,
    `xid`               VARCHAR(128) NOT NULL,
    `transaction_id`    BIGINT,
    `resource_group_id` VARCHAR(32),
    `resource_id`       VARCHAR(256),
    `branch_type`       VARCHAR(8),
    `status`            TINYINT,
    `client_id`         VARCHAR(64),
    `application_data`  VARCHAR(2000),
    `gmt_create`        DATETIME(6),
    `gmt_modified`      DATETIME(6),
    PRIMARY KEY (`branch_id`),
    KEY `idx_xid` (`xid`)
) ENGINE = InnoDB
  DEFAULT CHARSET = utf8mb4;

-- the table to store lock data
CREATE TABLE IF NOT EXISTS `lock_table`
```

我们为 Seata 配置一下注册中心和配置中心。

修改配置文件改为 Nacos 方式，修改 conf/registry.conf

```
 type = "nacos"

  nacos {
    application = "seata-server"
    serverAddr = "127.0.0.1:8848"
    group = "SEATA_GROUP"
    namespace = ""
    cluster = "default"
    username = "nacos"
    password = "nacos"
  }

再改一下下面的配置中心
config {
  # file、nacos 、apollo、zk、consul、etcd3
  type = "nacos"

  nacos {
    serverAddr = "127.0.0.1:8848"
    namespace = ""
    group = "SEATA_GROUP"
    username = "nacos"
    password = "nacos"
    dataId = "seataServer.properties"
  }
}
```

---

获取/seata/script/config-center/config.txt，修改配置信息

```
client.tm.degradeCheckPeriod=2000
store.mode=db
store.file.dir=file_store/data
store.file.maxBranchSessionSize=16384
store.file.maxGlobalSessionSize=512
store.file.fileWriteBufferCacheSize=16384
store.file.flushDiskMode=async
store.file.sessionReloadReadSize=100
store.db.datasource=druid
store.db.dbType=mysql
store.db.driverClassName=com.mysql.jdbc.Driver
store.db.url=jdbc:mysql://127.0.0.1:3306/seata?useUnicode=true
store.db.user=root
store.db.password=root
store.db.minConn=5
store.db.maxConn=30
store.db.globalTable=global_table
store.db.branchTable=branch_table
store.db.queryLimit=100
store.db.lockTable=lock_table
store.db.maxWait=5000
```

db模式存储

修改数据库相关配置

配置事务分组， 要与客户端配置的事务分组一致

#my_test_tx_group 需要与客户端保持一致 default 需要跟客户端和 registry.conf 中 registry 中的 cluster 保持一致 （客户端 properties 配置：spring.cloud.alibaba.seata.tx - service - group=my_test_tx_group）

事务分组： 异地机房停电容错机制

my_test_tx_group 可以自定义 比如：(guangzhou、shanghai...） ， 对应的 client 也要去设置

seata.service.vgroup - mapping.projectA=guangzhoudefault

必须要等于 registry.confi cluster = "default"

```
transport.shutdown.wait=3
service.vgroupMapping.my_test_tx_group=default
service.default.grouplist=127.0.0.1:8091
service.enableDegrade=false
```

配置事务分组名称

配置参数同步到 Nacos

shell:

```
sh ${SEATAPATH}/script/config-center/nacos/nacos-config.sh
-h localhost -p 8848 -g SEATA_GROUP
-t 5a3c7d6c-f497-4d68-a71a-2e5e3340b3ca
```

参数说明：

-h: host，默认值 localhost

-p: port，默认值 8848

-g: 配置分组，默认值为 'SEATA_GROUP'

```
chaos@DCL MINGW64 /f/Resource/seata/seata/script/config-center/nacos ((v1.4.0))
$ sh nacos-config.sh -h localhost
set nacosAddr=localhost:8848
set group=SEATA_GROUP
Set transport.type=TCP successfully
Set transport.server=NIO successfully
Set transport.heartbeat=true successfully
Set transport.enableClientBatchSendRequest=false successfully
Set transport.threadFactory.bossThreadPrefix=NettyBoss successfully
Set transport.threadFactory.workerThreadPrefix=NettyServerNIOWorker successfully
Set transport.threadFactory.serverExecutorThreadPrefix=NettyServerBizHandler successfully
Set transport.threadFactory.shareBossWorker=false successfully
Set transport.threadFactory.clientSelectorThreadPrefix=NettyClientSelector successfully
Set transport.threadFactory.clientSelectorThreadSize=1 successfully
Set transport.threadFactory.clientWorkerThreadPrefix=NettyClientWorkerThread successfully
Set transport.threadFactory.bossThreadSize=1 successfully
Set transport.threadFactory.workerThreadSize=default successfully
Set transport.shutdown.wait=3 successfully
Set service.vgroupMapping.my_test_tx_group=default successfully
Set service.default.grouplist=127.0.0.1:8091 successfully
```

精简配置

```
service.vgroupMapping.my_test_tx_group=default
service.default.grouplist=127.0.0.1:8091
service.enableDegrade=false
service.disableGlobalTransaction=false
store.mode=db
store.db.datasource=druid
store.db.dbType=mysql
store.db.driverClassName=com.mysql.jdbc.Driver
store.db.url=jdbc:mysql://127.0.0.1:3306/seata?useUnicode=true
store.db.user=root
store.db.password=root
store.db.minConn=5
store.db.maxConn=30
store.db.globalTable=global_table
store.db.branchTable=branch_table
store.db.queryLimit=100
store.db.lockTable=lock_table
store.db.maxWait=5000
```

启动 Seata Server

源码启动: 执行 server 模块下 io.seata.server.Server.java 的 main 方法

命令启动: bin/seata-server.sh -h 127.0.0.1 -p 8091 -m db -n 1 -e test

启动成功，默认端口 8091

## 3.6 Seata Client 快速配置

（1）添加 pom 依赖

```
<!-- seata-->
<dependency>
    <groupId>com.alibaba.cloud</groupId>
    <artifactId>spring-cloud-starter-alibaba-seata</artifactId>
</dependency>
```

（2）各微服务对应数据库中添加 undo_log 表

```
CREATE TABLE `undo_log` (
 `id` bigint(20) NOT NULL AUTO_INCREMENT,
 `branch_id` bigint(20) NOT NULL,
 `xid` varchar(100) NOT NULL,
 `context` varchar(128) NOT NULL,
 `rollback_info` longblob NOT NULL,
 `log_status` int(11) NOT NULL,
 `log_created` datetime NOT NULL,
 `log_modified` datetime NOT NULL,
 PRIMARY KEY (`id`),
 UNIQUE KEY `ux_undo_log` (`xid`,`branch_id`)
 ) ENGINE=InnoDB AUTO_INCREMENT=1 DEFAULT CHARSET=utf8;
```

（3）修改 register.conf,配置 nacos 作为 registry.type&config.type，对应 seata server 也使用 nacos。注意：需要指定 group = "SEATA_GROUP"，因为 Seata Server 端指定了 group = "SEATA_GROUP"，必须保证一致

```
registry {
  # file 、nacos 、eureka、redis、zk、consul、etcd3、sofa
  type = "nacos"
 nacos {
  serverAddr = "localhost"
  namespace = ""
  cluster = "default"
  group = "SEATA_GROUP"
 }
 }
 config {
  # file、nacos 、apollo、zk、consul、etcd3、springCloudConfig
  type = "nacos"

  nacos {
  serverAddr = "localhost"
  namespace = ""
  group = "SEATA_GROUP"
 }
 }
```

如果出现这种问题：

一般大多数情况下都是因为配置不匹配导致的：

1.检查现在使用的 seata 服务和项目 maven 中 seata 的版本是否一致

2.检查 tx-service-group，nacos.cluster，nacos.group 参数是否和 Seata Server 中的配置一致

跟踪源码：seata/discover 包下实现了 RegistryService#lookup，用来获取服务列表

```
NacosRegistryServiceImpl#lookup
String clusterName = getServiceGroup(key); #获取seata server集群名称
List<Instance> firstAllInstances =
getNamingInstance().getAllInstances(getServiceName(),
getServiceGroup(), cluste
rs)
```

（4）修改 application.yml 配置

配置 seata 服务事务分组，要与服务端 nacos 配置中心中 service.vgroup_mapping 的后缀对应

```
server:
  port: 8020

 spring:
  application:
  name: order-service
  cloud:
  nacos:
  discovery:
  server-addr: 127.0.0.1:8848
  alibaba:
  seata:
  tx-service-group:
  my_test_tx_group # seata 服务事务分组

  datasource:
  type: com.alibaba.druid.pool.DruidDataSource
  druid:
  driver-class-name: com.mysql.cj.jdbc.Driver
  url:
jdbc:mysql://localhost:3306/seata_order?useUnicode=true&character
Encoding=UTF-8&serverTimezone=Asia/Shanghai
  username: root
  password: root
  initial-size: 10
  max-active: 100
  min-idle: 10
  max-wait: 60000
  pool-prepared-statements: true
  max-pool-prepared-statement-per-connection-size: 20
  time-between-eviction-runs-millis: 60000
min-evictable-idle-time-millis: 300000
  test-while-idle: true
  test-on-borrow: false
  test-on-return: false
  stat-view-servlet:
  enabled: true
  url-pattern: /druid/*
  filter:
  stat:
  log-slow-sql: true
  slow-sql-millis: 1000
  merge-sql: false
```

（5）微服务发起者（TM 方）需要添加@GlobalTransactional 注解

```
@Override
 //@Transactional
 @GlobalTransactional(name="createOrder")
 public Order saveOrder(OrderVo orderVo){
  log.info("=============用户下单=================");
  log.info("当前 XID: {}", RootContext.getXID());

 // 保存订单
  Order order = new Order();
  order.setUserId(orderVo.getUserId());
  order.setCommodityCode(orderVo.getCommodityCode());
  order.setCount(orderVo.getCount());
  order.setMoney(orderVo.getMoney());
  order.setStatus(OrderStatus.INIT.getValue());

  Integer saveOrderRecord = orderMapper.insert(order);
  log.info("保存订单{}", saveOrderRecord > 0 ? "成功" : "失败");

 //扣减库存

storageFeignService.deduct(orderVo.getCommodityCode(),orderVo.get
Count());

 //扣减余额

accountFeignService.debit(orderVo.getUserId(),orderVo.getMoney())
;

 //更新订单
  Integer updateOrderRecord =
orderMapper.updateOrderStatus(order.getId(),OrderStatus.SUCCESS.g
etValue());
  log.info("更新订单 id:{} {}", order.getId(), updateOrderRecord >
0 ? "成功" : "失败");

 return order;

 }
```

测试

分布式事务成功，模拟正常下单、扣库存，扣余额

分布式事务失败，模拟下单扣库存成功、扣余额失败，事务是否回滚

# 实验十 云计算与云部署

## 一、 目的与要求

了解 WSL 和虚拟机的概念； 掌握 docker 的安装与使用； 了解 devops，以及 kubernetes、jenkins。

https://www.docker.com/

https://kubernetes.io/zh-cn/docs/home/

https://www.jenkins.io/zh/

## 二、 实验内容

2.1 了解 WSL 和 vmware，安装 Linux

2.2 安装 docker，学习 docker 的基本概念和基本命令

2.3 在 docker 上部署微服务项目

2.3 通过官网文档，了解 kubernetes

2.4 了解 jenkins

## 三、 实验指导

1、WSL：https://learn.microsoft.com/zh-cn/windows/wsl/

这是微软官方关于 wsl 的文档，包括其下载、安装、使用方法。同学们可参考自行安装即可。WSL 教程这是我整理的 WSL 教程，大家可参考着学习。

2、docker

- docker 官网：https://www.docker.com/get-started/
- WSL 2 上的 Docker 远程容器入门：

https://learn.microsoft.com/zh-cn/windows/wsl/tutorials/wsl-containers

- 菜鸟教程：https://www.runoob.com/docker/docker-tutorial.html
- 【GeekHour】30 分钟 Docker 入门教程（这个非常不错）

https://www.bilibili.com/video/BV14s4y1i7Vf/?p=6&spm_id_from=pageDriver&vd_source=c5e65f0

20919c07bb4d1cc902ee3c1d7

- 实验平台 play-with-docker：https://labs.play-with-docker.com/
- 文档：https://docker.easydoc.net/（可以用来参考做课件）配套视频：https://www.bilibili.com/video/BV11L411g7U1/?vd_source=c5e65f020919c07bb4d1cc902ee3c1d7

实验步骤：

(1) 在 Windows 上安装 Docker Desktop：https://docs.docker.com/desktop/install/windows-install/

(2) 在 WSL：Ubuntu 中就可以运行 docker 命令了，学习下 docker 基本命令：

- docker version
- docker info

Docker 是一个用于创建、部署和管理容器化应用程序的开源平台。以下是一些 Docker 的基本常用命令：

1. 检索镜像：使用 docker pull 命令从 Docker Hub 或其他注册中心拉取镜像。
2. 查看本地镜像：使用 docker images 命令列出本地存储的所有镜像。
3. 创建容器：使用 docker run 命令创建并启动一个新的容器。
4. 查看运行中的容器：使用 docker ps 命令显示当前运行的容器。
5. 停止容器：使用 docker stop 命令停止一个运行中的容器。
6. 启动已停止的容器：使用 docker start 命令启动一个已停止的容器。
7. 删除容器：使用 docker rm 命令删除一个已停止的容器。
8. 查看容器日志：使用 docker logs 命令查看容器的日志。
9. 进入容器：使用 docker exec 命令进入一个运行中的容器。
10. 保存和加载镜像：使用 docker save 和 docker load 命令将镜像保存为文件或从文件加载镜像。
11. 构建镜像：使用 docker build 命令根据 Dockerfile 构建新的镜像。
12. 推送镜像到注册中心：使用 docker push 命令将本地镜像推送到 Docker Hub 或其他注册中心。

(3) 构建 Java 映像：https://docs.docker.com/language/java/build-images/

下面这个视频非常好，这也是我们学习的**重要目标**：【狂神说：SpringBoot 微服务打包 Docker 镜像】

https://www.bilibili.com/video/BV1og4y1q7M4/?p=39&spm_id_from=pageDriver&vd_source=c5e65f020919c07bb4d1cc902ee3c1d7

3、kubernetes

https://kubernetes.io/zh-cn/docs/setup/

4、jenkines

https://www.jenkins.io/zh/